

Scuola di Scienze Matematiche, Fisiche e Naturali

Laurea Magistrale in Data Science, Calcolo Scientifico & Intelligenza Artificiale

Master's Thesis

DESIGN AND ANALYSIS OF ALGORITHMS AND DATA STRUCTURES FOR ANISOTROPIC HIERARCHICAL SPLINE REFINEMENT

COSIMO MATASSINI

Advisor: *Prof. Carlotta Giannelli*

Co-Advisor: *Dr. Mattia Matucci*

Academic Year 2025-2026

Cosimo Matassini: *Design and analysis of algorithms and data structures for anisotropic hierarchical spline refinement,* Laurea Magistrale in Data Science, Calcolo Scientifico & Intelligenza Artificiale, © Academic Year 2025-2026

Acknowledgements

I would like to express my deepest gratitude to my advisor Carlotta Giannelli, co-advisor Mattia Matucci, and Ernesto Palchetti for their guidance, technical feedback, and continuous academic support throughout the development of this thesis.

A profound thank you goes to my family, whose unconditional support and constant presence have been my greatest strength during these years of study.

Finally, my warmest appreciation goes out to everyone who has believed in me, stood by my side, and shared in the doubts, steps, and successes of this journey. This achievement belongs to you as well.

Abstract

Adaptive spline approximation aims to concentrate spatial resolution exclusively where it is needed, preserving the local support property of tensor-product B-spline bases. Classical hierarchical B-splines (HB-splines) achieve local refinement through a totally ordered sequence of globally nested spaces. While effective for isotropic features, this approach couples all coordinate directions, forcing uniform bisections along all spatial axes. To overcome this operational restriction, Lattice Hierarchical B-splines (LHB-splines) replace the traditional single-chain hierarchy with a partially ordered lattice of directional level vectors, enabling axis-selective anisotropic refinement. This thesis presents a complete algorithmic and data-structure pipeline for anisotropic hierarchical spline refinement. To represent directional dyadic splits efficiently, a normalized omnitree data structure—provided through the DyAda framework—is implemented and analyzed. Its topological efficiency, tree depth, and branching behavior are systematically compared against a reference restricted cyclic midpoint bintree (k-d tree), whose split axes are strictly coupled to node depth. The active leaves of the omnitree are mapped onto directional spline levels. The resulting classic hierarchical and lattice hierarchical active bases are evaluated within an adaptive discrete least-squares fitting framework for bivariate target functions. Structural comparisons show that prescribed cyclic split orders introduce substantial auxiliary subdivisions, whereas the omnitree maintains a minimal topology by avoiding forced orthogonal cuts. Numerical approximation experiments demonstrate the clear advantage of lattice hierarchical refinement across all tested scenarios. By successfully decoupling directional resolution, anisotropic refinement consistently outperforms standard hierarhical refinement, isolating local directional variations to achieve lower approximation errors with significantly fewer degrees of freedom. By integrating normalized omnitrees with partially nested spline spaces, this study confirms the practical viability and robust efficiency of directional hierarchical refinement, establishing it as a consistently superior framework for multidimensional applications.

CONTENTS

1 Introduction 13
2 Preliminaries 17
  2.1 B-spline models and hierarchical extensions 17
    2.1.1 Univariate B-splines 17
    2.1.2 Tensor-product B-splines 20
    2.1.3 Hierarchical B-splines 22
  2.2 Data structures for anisotropic mesh refinement 26
    2.2.1 k-d trees 27
    2.2.2 Hyperoctrees 30
    2.2.3 Omnitrees 33
3 Partially nested hierarchical B-spline spaces 37
  3.1 Lattice splines on adaptive meshes 38
    3.1.1 From isotropic to anisotropic hierarchical refinement 38
    3.1.2 Lattice hierarchy 39
    3.1.3 Lattice hierarchical B-splines 42
  3.2 Compatible meshes and linear independence 45
    3.2.1 Elimination of ghost functions 46
    3.2.2 Comparable marking 47
4 Algorithms and data structures for LHB-splines 49
  4.1 Python implementation of k-d trees and omnitrees 49
    4.1.1 Construction and refinement of the DyAda discretization 50
    4.1.2 Explicit representation of the cyclic k-d tree 51
    4.1.3 Conversion used by the comparison and statistics collected 53
  4.2 Comparison 54
    4.2.1 Protocol and interpretation of the data 54
    4.2.2 Illustrative two-dimensional example 55
    4.2.3 Strongly anisotropic two-dimensional refinement 55
    4.2.4 Fair random meshes 58
    4.2.5 Mesh produced by anisotropic function approximation 59
    4.2.6 Unrestricted random meshes in three and four dimensions 60

4.2.7 Overall assessment 61
4.3 Representation of lattice hierarchical meshes and LHB-splines 63
4.3.1 Construction pipeline 63
4.3.2 Knot vectors and tensor-product bases 64
4.3.3 Projection of the adaptive mesh onto a basis grid 65
4.3.4 Support of a candidate B-spline 66
4.3.5 Activity conditions 66
4.3.6 Active-function collection and output 67
5 Adaptive Approximation with LHB-Splines 69
5.1 Adaptive Spline Refinement 70
5.2 Error Estimation and Anisotropic Refinement 71
5.2.1 Least-squares fit and reported error 71
5.2.2 How an element is marked 72
5.2.3 How LHB-spline refinement directions are selected 72
5.2.4 Stopping the adaptive loop 73
5.3 Numerical Examples 73
5.3.1 First bivariate case: mixed directional features 74
5.3.2 Second bivariate case: an axis-aligned ridge 75
5.3.3 Overall interpretation 78
6 Conclusions 83
6.1 Key Findings 83
6.2 Impact and Future Directions 84

LIST OF FIGURES

Figure 2.1 Example of B-splines defined over the same knot vector [0 1 2 3 4 5 6 7] with different spline degrees, 1, 2 and 3 respectively. 18

Figure 2.2 Univariate B-spline bases, the first knot vector being [0 0 0 1.7 7 11] and the second one [0 0 1 2 3 4 4] 22

Figure 2.3 Tensor product B-splines, from $N_{\{0,0\},\{2,2\}}$ (upper left) to $N_{\{2,3\},\{2,2\}}$ (lower right) 22

Figure 2.4 Knot insertion for $p = 2$ B-splines. Inserting knots at $x = 1, 3, 5$ into the initial knot vector [0 0 0 2 4 6 6 6] refines the basis functions and narrows their support while maintaining overall continuity. 24

Figure 2.5 Hierarchical refinement in a one-dimensional setting. 25

Figure 2.6 Tensor product refinement (*left*) and hierarchical refinement (*right*) 25

Figure 2.7 Mesh subdivision and its corresponding binary 2-d tree 28

Figure 2.8 Mesh subdivision and its corresponding hyperoctree (quadtree). 31

Figure 2.9 Mesh subdivision and its corresponding normalized omnitree. The leaves are the only nodes where their binary refinement label contains only zeros; they are omitted to indicate the corresponding rectangular mesh subdivision shown on the left. 34

Figure 2.10 Flat vector representation obtained from the preorder traversal of the tree structure in Figure 2.9. 36

Figure 3.1 $f(x, y) = \frac{x}{20} - \frac{\sin(y^2)}{10}$ 39

Figure 3.2 Lattice hierarchy with $f_{max} = 4$. 41

Figure 3.3 Geometric interpretation of the LHB-spline activation rule as described in 3. 43

Figure 3.4 Geometric configuration of active LHB-spline basis functions. The support (dashed line) contains an active cell at its own level k in (a), whereas in (b) it is entirely covered by finer cells of descendant levels, making it a ghost function. 44
Figure 4.1 Two hierarchical representations of the illustrative 2-dimensional mesh considered in Section 4.2.2. The adaptive mesh (a) is shown together with corresponding k-d tree (b) and omnitree (c). Leaf colors identify corresponding geometric cells. 56
Figure 4.2 Strongly anisotropic two-dimensional refinement considered in Section 4.2.3. The omnitree (a) and k-d tree (b) meshes are shown. Only leaf boundaries are drawn; colors and cell labels have been removed to make the additional k-d tree cuts visible. 57
Figure 4.3 Fair random two-dimensional case considered in Section 4.2.4 with 501 leaves in both tree structures. The leaf partitions coincide 58
Figure 4.4 Adaptive meshes associated with the anisotropic function-approximation case considered in Section 4.2.5. Fine structures become visually dense at this scale, but the different amount of subdivisions remains apparent. 60
Figure 5.1 First numerical example function 5.3 75
Figure 5.2 Evolution of the least-squares approximation for the mixed-feature surface. The first row shows HB-spline refinement and the second row LHB-spline refinement. 76
Figure 5.3 Evolution of the active meshes for the mixed-feature surface. HB refinement produces square cells, whereas the LHB-spline mesh contains elongated cells where the directional indicator selects a single component. 77
Figure 5.4 Evolution of the sampled absolute error for the mixed-feature surface. Each panel uses the color scale generated for that iteration. 78
Figure 5.5 MSE against degrees of freedom for the first surface-fitting experiment. 78
Figure 5.6 Second numerical example function 5.4 79

Figure 5.7 Evolution of the least-squares approximation for the axis-aligned ridge. 80
Figure 5.8 Evolution of the active meshes for the axis-aligned ridge. The LHB-spline mesh concentrates thin vertical cells around the transition region. 81
Figure 5.9 Evolution of the sampled absolute error for the axis-aligned ridge. 81
Figure 5.10 MSE against degrees of freedom for the axis-aligned ridge. 82

"Remembering that you are going to die is the best way I know to avoid the trap of thinking you have something to lose."
— Steve Jobs

1

INTRODUCTION

Splines are a fundamental class of piecewise polynomial functions widely used for functional approximation and geometric modeling due to their high continuity, numerical stability, and local support properties. When extending univariate B-splines to multidimensional domains, the standard approach relies on tensor-product constructions. Although tensor-product B-splines offer a straightforward formulation, they suffer from a major operational limitation: inserting a knot along one coordinate direction propagates the refinement globally across the entire parametric domain. Consequently, when a function exhibits localized features or rapid variations in a small region, tensor-product meshes force unnecessary subdivisions throughout the computational domain, introducing many superfluous degrees of freedom (DoFs) and significantly increasing memory consumption and computational costs. To overcome this limitation, local mesh refinements allow resolutions to be updated exclusively where additional detail is required. Hierarchical B-splines (HB-splines) address localized refinement by organizing nested sequences of spaces, replacing coarse basis functions with finer ones in targeted subdomains. However, classical HB-splines enforce isotropic refinement, dyadically splitting all coordinate directions simultaneously. This coupling turns out to be highly inefficient for directional features, such as sharp ridges, where rapid variations occur along a single coordinate. To solve this, lattice hierarchical B-splines (LHB-splines) [6] replace the traditional single-chain hierarchy with a partially ordered lattice hierarchy. This allows for axis-selective anisotropic refinement and the decoupling of spatial resolutions. Beyond LHB-splines, several alternative anisotropic spline paradigms exist in the literature, including Truncated Hierarchical B-splines (THB-splines) [19], T-splines [39], Locally Refined B-splines (LR B-splines) [13], and Patchwork B-splines (PB-splines) [15, 20].

Within hierarchical space partitioning for mesh refinement, three data structures play a central role: k-d trees, hyperoctrees and omnitrees. Orig-

inally introduced by Bentley in 1975 for efficient associative search [4], the k-dimensional tree, or k-d tree, is a versatile binary space-partitioning data structure. Although traditionally utilized in computational geometry for point cloud processing [5] and nearest-neighbor searches [18, 29], a specific variant, often referred to as the region-based k-d tree or adaptive binary tree [37], has become highly valuable in the context of numerical simulations and mesh generation. Its hierarchical binary structure makes it suitable for representing recursively refined spatial partitions in a variety of applications, including adaptive mesh refinement in a d-dimensional domain $\Omega \subset \mathbb{R}^d$.

Among the most widely used hierarchical spatial data structures are *quadtrees* [16] and *octrees* [26]. These structures recursively partition the computational domain into geometrically regular cells, generating a tree whose topology directly reflects the local refinement of the space. Quadtrees and octrees are special cases of a more general family known as *hyperoctrees*. While quadtrees operate in two-dimensional domains [35] and octrees in three-dimensional domains [36], hyperoctrees extend the same recursive subdivision principle to arbitrary dimensions.

The hierarchical structures presented so far represent two opposite approaches to adaptive spatial subdivision. Hyperoctrees perform isotropic refinement by bisecting every coordinate direction simultaneously, whereas k-d trees refine only one direction at each level. Although both approaches are effective in many applications, neither provides complete flexibility when the solution exhibits anisotropic features. Refinement of all directions may generate many unnecessary elements, while refinement of a single direction often leads to unnecessarily deep trees. In general, k-d trees tend to be narrow but deep, whereas hyperoctrees are usually shallow but wide. Omnitrees [33] were introduced as a generalization of both hyperoctrees and binary refinement trees, combining their respective advantages while avoiding their main limitations, allowing them to minimize both tree depth and width simultaneously. Their defining characteristic is that each refinement step may bisect an arbitrary subset of the coordinate directions. Consequently, an omnitree naturally supports both isotropic and anisotropic refinement within the same hierarchical structure.

The primary objective of this thesis is to design, implement and analyze a complete computational framework for anisotropic hierarchical spline refinement by integrating lattice hierarchical B-splines with the omnitree structure, the latter provided through the `DyAda` framework [32]. To quantify the structural benefits of omnitrees, their number of leaves

and total nodes, and average leaf depth are compared against the k-d tree structure across different dimensional settings. In addition to this, an adaptive least-square fitting framework was developed, and through numerical experiments, the thesis wants to show that anisotropic refinement can achieve approximation accuracy while requiring significantly fewer degrees of freedom than isotropic refinement.

The remainder of this thesis is organized into five chapters: Chapter 2 reviews univariate and multivariate B-splines, classical hierarchical B-splines and detailed explanations of k-d trees, hyperoctrees and omnitrees. Chapter 3 presents the theory of lattice hierarchical B-splines, covering active basis selection, mesh compatibility, ghost function deactivation, and comparable marking. Chapter 4 shows the software implementation and the topological comparisons between k-d trees and omnitrees. Finally, Chapter 5 formulates the adaptive least-squares fitting loop and presents numerical bivariate experiments.

2

PRELIMINARIES

This chapter reviews the theoretical concepts that constitute the foundation of the methods presented in the remainder of this thesis. Rather than providing an exhaustive treatment, the objective is to introduce the mathematical and computational tools necessary to understand the proposed approach.

Several figures presented in this chapter were generated using custom Python scripts developed specifically for this work. The complete source code is publicly available in the GitHub repository at [9].

2.1 B-SPLINE MODELS AND HIERARCHICAL EXTENSIONS

This section introduces the mathematical foundations of B-spline-based geometric representations. It begins with the definition of univariate B-splines and their fundamental properties, including the knot vector, basis functions, and continuity. The discussion is then extended to multivariate spline spaces through tensor-product B-splines, which provide a straightforward construction of surfaces and higher-dimensional models. Finally, hierarchical B-splines are presented as an extension of the tensor-product framework, enabling local refinement while preserving the desirable properties of the underlying B-spline basis.

2.1.1 *Univariate B-splines*

B-splines are piecewise polynomial basis functions that provide a flexible and numerically stable framework for constructing smooth curves and surfaces over a prescribed knot vector [12]

$$\mathbf{t} = \{t_0, \ldots, \underbrace{t_p, \ldots, t_{n+1}}_{\boldsymbol{\tau}=\{\tau_0,\ldots,\tau_L\}}, \ldots, t_{n+p+1}\} \tag{2.1}$$

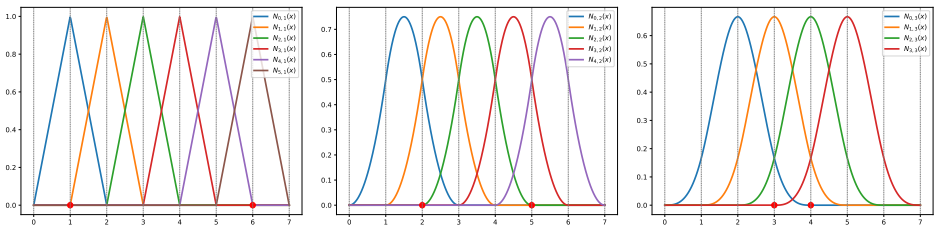


Figure 2.1: Example of B-splines defined over the same knot vector [0 1 2 3 4 5 6 7] with different spline degrees, 1, 2 and 3 respectively.

with $t_i \in \mathbb{R}$ and $t_i \leqslant t_{i+1},\ i = 0, \dots, n+p$.

In the knot vector representation at 2.1, the polynomial pieces are connected at specific parameter values called *breakpoints*, which are determined by the knot vector itself. Since knots can be repeated multiple times, the breakpoint vector $\boldsymbol{\tau}$ contains only the distinct knot values over which the spline is represented by individual polynomial pieces. These breakpoints define L parametric intervals, namely the intervals between consecutive values of $\boldsymbol{\tau}$. The first and last breakpoints, τ_0 and τ_L, define the effective parametric domain where the spline is evaluated: $\Omega = [\tau_0, \tau_L]$.

The knot values outside this interval, which appear in the knot vector, are used to construct the basis functions and determine their support, but they do not introduce additional spline segments.

Definition 2.1.1 (Univariate B-spline basis)**.** Given a polynomial degree $p \geqslant 0$ and a knot vector $\mathbf{t} = \{t_0, t_1, \dots, t_{n+p+1}\}$ as defined in (2.1), the *univariate B-spline basis* of degree p associated with $\mathbf{t}$ is the set of $n+1$ piecewise polynomial functions defined as:

$$\mathcal{B}_{\mathbf{t},p} := \left\{N_{i,p}\right\}_{i=0}^{n} \tag{2.2}$$

where each $N_{i,p} : \Omega \to \mathbb{R}$ denotes the i-th B-spline basis function.

Each B-spline, denoted by $N_{i,p}\ \forall i = 0, \dots, n$ has support on the interval $[t_i, t_{i+p+1}]$: consequently, each basis function is nonzero over at most $p+1$ consecutive knot intervals and vanishes elsewhere. In 2.1, n denotes the highest index of the B-spline basis functions. An example of this local support property is shown in Figure 2.1.

The degree of a spline, denoted by p, determines the complexity of the polynomial pieces that compose the spline and plays a fundamental role in its approximation and smoothness properties; it consists of piecewise polynomial segments of degree p. Over each knot span, exactly $p+1$

B-spline basis functions are simultaneously non-zero. Thus, the spline degree determines both the flexibility of the resulting curve and the extent of its local support.

Increasing the spline degree enables the curve to represent more complex geometric features and generally provides a higher level of smoothness, since higher-degree polynomials can capture more intricate variations in the underlying shape. However, lower-degree splines offer several practical advantages. They are computationally more efficient and generally exhibit better numerical stability than higher-degree splines. Moreover, they provide stronger local control, meaning that modifying a control point affects only a limited portion of the curve, thereby simplifying curve editing and manipulation. Finally, lower-degree splines reduce the risk of overfitting, as higher-degree polynomial segments may adapt excessively to noise in the data rather than capturing the underlying trend.

As mentioned previously, each knot can be repeated multiple times: each one has its own multiplicity m_i, and that determines the continuity of the spline at that knot, which is C^{p-m_i}. This relation shows that increasing its multiplicity decreases the smoothness of the curve at the corresponding breakpoint.

The effect of knot multiplicity can be understood intuitively by considering a few special cases:

If a knot has multiplicity ($m_i = 1$) (a simple knot), the spline achieves its maximum continuity, that is C^{p-1}. The adjacent polynomial segments blend together smoothly, producing a highly regular curve. If the multiplicity is increased, continuity is progressively reduced. When the continuity of a knot reaches the degree of the spline, it becomes C^0 continuous, meaning that the curve remains connected but its first derivative is discontinuous. When the multiplicity attains $m_i = p + 1$, the continuity becomes C^{-1}: in this case, the polynomial pieces are no longer connected, resulting in a curve break.

Knot multiplicity is therefore an essential design parameter in B-splines. By appropriately repeating knots, it is possible to locally reduce the smoothness of the curve without affecting the remaining portions of the spline. This property is particularly useful in geometric modeling and approximation problems, where certain regions may require sharp corners, changes in curvature, or interpolation constraints while preserving a smooth behavior elsewhere. Consequently, non-uniform knot vectors, where the knot intervals have different lengths, provide significantly

greater flexibility than uniform ones, allowing local control over the shape.

In practical applications, special classes of knot vectors are commonly used. A knot vector is said to be *clamped* or *open* if its first and last knot values each have multiplicity $p + 1$:

$$t_0 = t_1 = \cdots = t_p = \tau_0, \tag{2.3}$$

and

$$t_{n+1} = t_{n+2} = \cdots = t_{n+p+1} = \tau_L. \tag{2.4}$$

The distribution of breakpoints has a direct impact on the geometric properties of the splines, as it determines where the polynomial pieces meet and how smoothly they are connected.

For a clamped knot vector whose internal knots are strictly increasing, that is $t_p < t_{p+1} < \cdots < t_{n+1}$, the number of intervals is always equal to $L = n + 1 - p$. Hence, the number of B-splines matches the number of intervals plus the degree of the spline.

However, in the general case, this relationship is no longer sufficient because knots may be repeated. When a knot appears multiple times in the knot vector, its multiplicity affects both the number of distinct breakpoints and the continuity of the spline at that location.

In Figure 2.1, different degree behaviors are shown with the same knot vector [0 1 2 3 4 5 6 7]: as the degree increases, the number of B-splines is reduced and the red dots, identifying τ_0 and τ_L respectively, get closer to each other, due to 2.3 and 2.4.

2.1.2 *Tensor-product B-splines*

The univariate B-spline basis introduced in the previous section can be naturally extended to multivariate domains through the tensor-product construction [31].

Given a collection of univariate spline spaces $V_1, V_2, \ldots, V_d$, defined over the corresponding knot vectors, the associated multivariate spline space is obtained through the tensor product

$$V = V_1 \otimes V_2 \otimes \cdots \otimes V_d$$

where d represents the number of dimensions the spline space V is defined on.

The result consists of all functions that can be expressed as products of univariate spline basis functions. Each spline space along the j-th coordinate direction is generated by the univariate basis

$$\mathcal{B}^{(j),p_j}_{\mathbf{t}^{(j)}} = \left\{ N^{(j)}_{i_j,p_j} \right\}_{i_j=0}^{n_j}.$$

The multivariate tensor-product basis is then obtained by multiplying the corresponding univariate basis functions along each coordinate direction and it is defined as follows.

Definition 2.1.2 (Multivariate Tensor-Product B-spline Basis)**.** Let $d \geqslant 1$ be the parametric dimension, $\mathbf{p} = (p_1, \ldots, p_d)$ the vector of polynomial degrees, and $\mathbf{T} = \{\mathbf{t}^{(1)}, \ldots, \mathbf{t}^{(d)}\}$ the collection of knot vectors for each coordinate direction. The *multivariate tensor-product B-spline basis* associated with $\mathbf{T}$ and $\mathbf{p}$ is the set of functions:

$$\mathcal{B}_{\mathbf{T},\mathbf{p}} := \left\{ N_{\mathbf{i},\mathbf{p}}(\mathbf{x}) \;\middle|\; i_j = 0, \ldots, n_j,\ j = 1, \ldots, d \right\}, \tag{2.5}$$

where each multivariate B-spline basis function $N_{\mathbf{i},\mathbf{p}} : \Omega \to \mathbb{R}$ evaluated at $\mathbf{x} = (x_1, \ldots, x_d) \in \mathbb{R}^d$ is defined as the product of the corresponding univariate basis functions:

$$N_{\mathbf{i},\mathbf{p}}(\mathbf{x}) := \prod_{j=1}^{d} N^{(j)}_{i_j,p_j}(x_j), \tag{2.6}$$

with $\mathbf{i} = (i_1, \ldots, i_d)$ denoting the multi-index identifying the basis function across the directions.

The collection of knot vectors $\mathbf{T}$ naturally partitions the parametric domain Ω into a cartesian grid, which forms the parametric mesh. The individual non-empty cells of this grid, defined by the sub-intervals between consecutive distinct knots in each coordinate direction, are the elements of the mesh. The union of all element closures fully covers the parametric domain Ω.

For instance, Figure 2.3 shows the case of a bivariate tensor-product setting where two different knot vectors are defined for each direction, whose representation is shown in Figure 2.2. The continuity across element boundaries is determined independently along each parameter direction by the degree and the multiplicity of the corresponding knots.

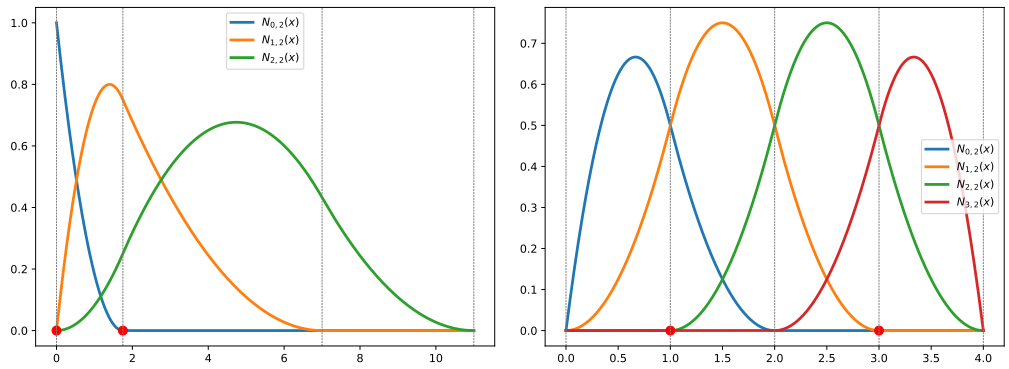


Figure 2.2: Univariate B-spline bases, the first knot vector being [0 0 0 1.7 7 11] and the second one [0 0 1 2 3 4 4]

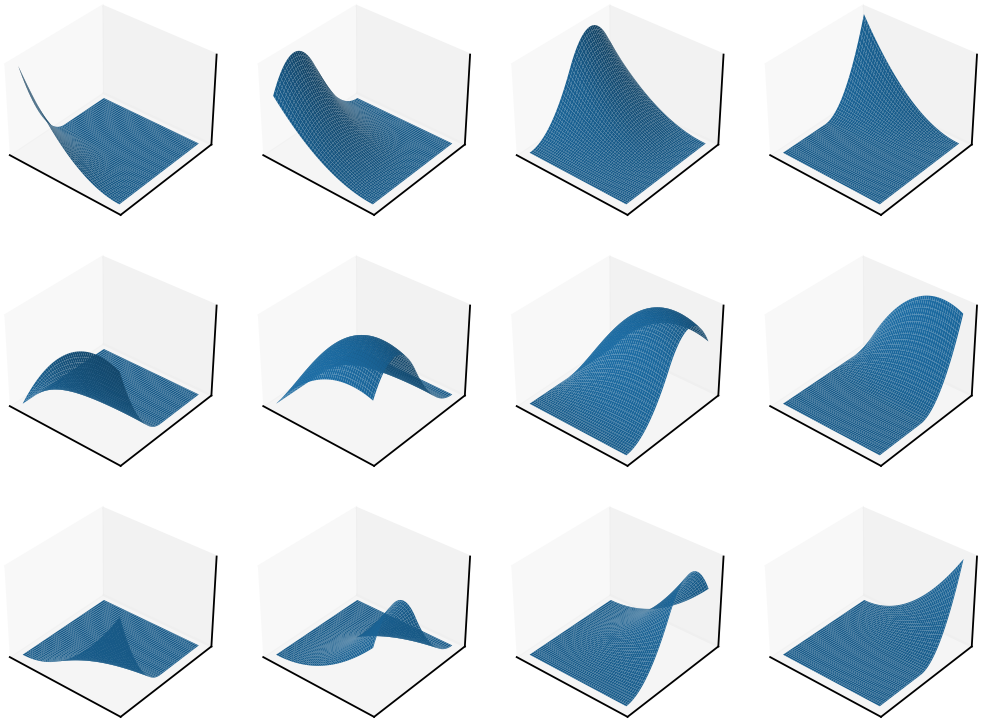


Figure 2.3: Tensor product B-splines, from $N_{\{0,0\},\{2,2\}}$ (upper left) to $N_{\{2,3\},\{2,2\}}$ (lower right)

2.1.3 *Hierarchical B-splines*

Tensor-product spline spaces exhibit an important limitation. Refinement is performed independently along each coordinate direction, implying that inserting a knot propagates across the entire parameter domain

in that direction. Consequently, refining only a small localized region requires the insertion of additional basis functions over the whole tensor-product grid, introducing many unnecessary degrees of freedom, meaning the free parameters (the coefficients or weights of the basis functions) that determine the shape and behavior of the spline.

Minimizing their number is highly desirable for several key reasons:

- Computational Efficiency: A smaller number of degrees of freedom drastically reduces memory storage requirements and CPU computational time for solving linear systems, evaluating geometry, and performing optimization or simulation tasks, such as in Finite Element Analysis (FEA) [30] or Isogeometric Analysis (IGA) [10, 22].
- Local Control and Precision: Avoiding unnecessary degrees of freedom prevents the introduction of superfluous geometric complexity, allowing resolution (and thus refinement) to be focused strictly where it is needed without over-refining the entire domain.
- Numerical Stability: Redundant or excessive degrees of freedom can lead to ill-conditioning in stiffness matrices [20] or geometric fitting problems, which introduces unwanted oscillations or numerical instabilities in calculations.

This lack of local refinement capability becomes increasingly restrictive in large-scale geometric modeling and numerical simulations. Hierarchical spline constructions address this limitation by allowing refinement to be performed only where additional resolution is required, while preserving the favorable approximation properties of tensor-product B-splines.

In fact, when it is necessary to refine the spline space in some regions of the domain, in order to better capture local features of the solution new knots are inserted in the knot vector. In a univariate setting, this is straightforward using the knot insertion algorithm, as shown in Figure 2.4.

An overview of the 1D hierarchical refinement strategy is illustrated in Figure 2.5. Initially, a set of uniform coarse B-spline basis functions is established across the entire computational domain Ω^0 at the coarsest level $\ell = 0$ to construct the global base space (Figure 2.5a). To increase local resolution, a targeted sub-domain $\Omega^1 \subseteq \Omega^0$ is selected where basis functions with reduced support are generated at level $\ell = 1$ (Figure 2.5b). Finally, the hierarchical basis is assembled by combining these fine functions with the remaining coarse ones, while the coarse basis functions

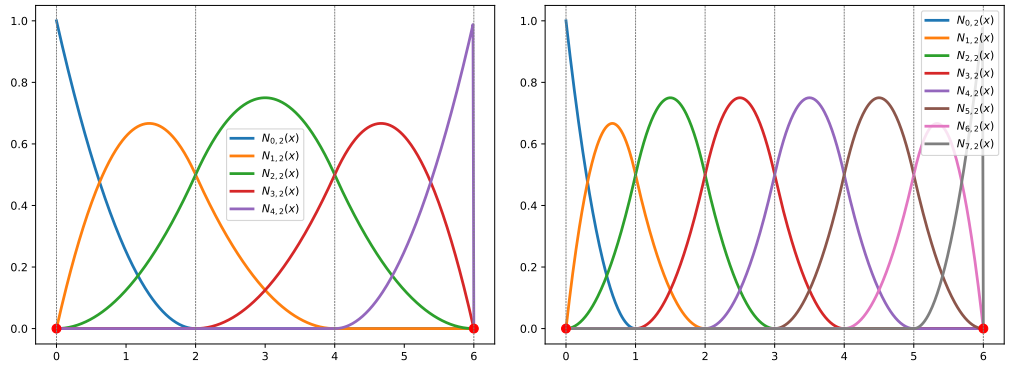


Figure 2.4: Knot insertion for $p = 2$ B-splines. Inserting knots at $x = 1, 3, 5$ into the initial knot vector [0 0 0 2 4 6 6 6] refines the basis functions and narrows their support while maintaining overall continuity.

whose support is entirely covered by the refined area are deactivated (represented by dashed curves) (Figure 2.5c).

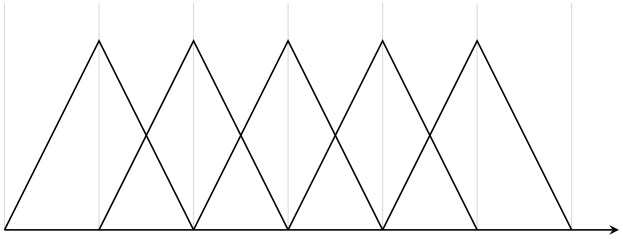
Because of the tensor-product structure, in a multidimensional setting simple refinements extend to the edges of the mesh, making local refinement impossible (see Figure 2.6, left). To avoid this, a hierarchical refinement approach is taken, where the basis functions in a selected refinement region are replaced with finer scale ones of the same type and leaving other regions coarser.

For simplicity and because of computational cost, each interval is divided into two equal parts at each refinement step (dyadic refinement). This way, each element is going to be split into 2^d elements at each refinement step. An example of hierarchical refinement is shown in Figure 2.6, right.

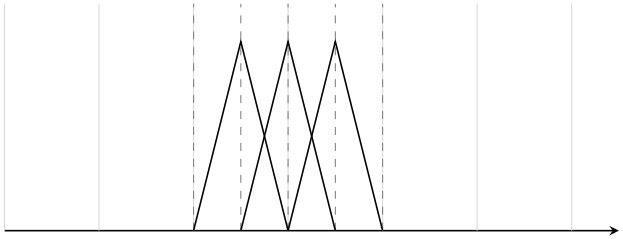
Consider a sequence of nested spline spaces, that is a grid of increasing resolution from the coarsest to the finest one: $V^0 \subset V^1 \subset \cdots \subset V^{N-1}$. Each spline space $V^i = \text{span}(\mathcal{B}^i)$ is spanned by the given tensor-product B-spline basis $\mathcal{B}^i$.

The areas of the entire domain Ω^0 that need to be refined are subdomains of Ω^0 itself: $\Omega^0 \supseteq \Omega^1 \supseteq \cdots \supseteq \Omega^{N-1}$, where Ω^i is going to be represented by a union of patches from the grid defined by the spline space V^i. The hierarchical B-spline basis $\mathcal{H}$ is going to be constructed by selecting functions from each level's B-spline basis $\mathcal{B}^i$ that have support in the domain Ω^i but not in the next finer domain Ω^{i+1}. Formally, the hierarchical B-spline basis is defined recursively [25, 40] as follows:

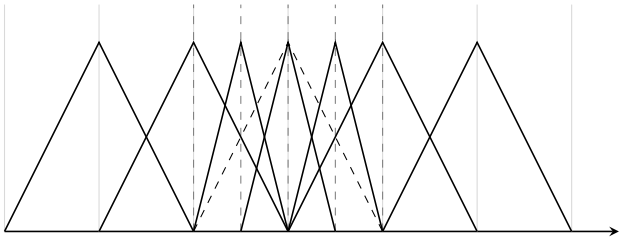
Definition 2.1.3 (Hierarchical B-spline Basis)**.** Starting from the coarsest level, the basis $\mathcal{H}$ is constructed through the following steps:



(a) Initial set of B-spline basis functions of degree $p = 1$ defined over the knot vector [0 1 2 3 4 5 6], with the global domain being Ω^0 at resolution level $\ell = 0$.



(b) Finer-level basis functions of degree $p = 1$ defined over the knot vector [2 2.5 3 3.5 4], with the sub-domain being $\Omega^1 \subseteq \Omega^0$ at resolution level $\ell = 1$



(c) Union of retained coarse functions and fine functions, omitting the deactivated coarse ones (dashed).

Figure 2.5: Hierarchical refinement in a one-dimensional setting.

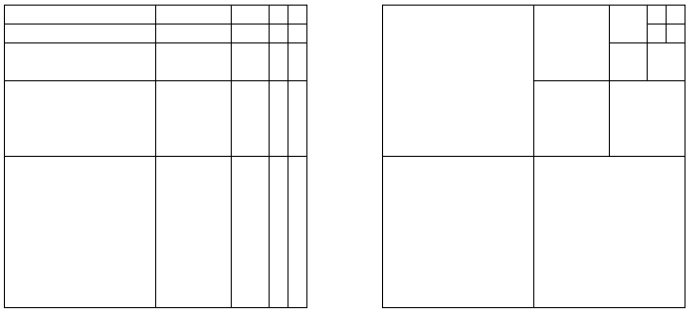


Figure 2.6: Tensor product refinement (*left*) and hierarchical refinement (*right*)

(I) Initialization: The basis at level 0 consists of all B-splines from the initial mesh with non-empty support:

$$\mathcal{H}^0 = \{\beta \in \mathcal{B}^0 : \text{supp } \beta \neq \emptyset\}.$$

(II) Recursion: For each level $\ell = 0, \dots, N-2$, the set $\mathcal{H}^{\ell+1}$ is defined as the union $\mathcal{H}^{\ell+1} = \mathcal{H}_A^{\ell+1} \cup \mathcal{H}_B^{\ell+1}$, where:

- $\mathcal{H}_A^{\ell+1}$ retains functions from the previous level that are not entirely contained within the refined subdomain $\Omega^{\ell+1}$:

$$\mathcal{H}_A^{\ell+1} = \{\beta \in \mathcal{H}^{\ell} : \operatorname{supp} \beta \not\subseteq \Omega^{\ell+1}\},$$

- $\mathcal{H}_B^{\ell+1}$ selects new functions from level $\ell+1$ whose support is entirely contained in $\Omega^{\ell+1}$:

$$\mathcal{H}_B^{\ell+1} = \{\beta \in \mathcal{B}^{\ell+1} : \operatorname{supp} \beta \subseteq \Omega^{\ell+1}\}.$$

(III) Final Basis: The hierarchical B-spline basis $\mathcal{H}$ is the set obtained at the finest level:

$$\mathcal{H} = \mathcal{H}^{N-1}.$$

This system defines an iterative filtering and updating process across N levels, to construct a final set $\mathcal{H}$. Initially, the set $\mathcal{H}^0$ consists of all functions β from the initial basis $\mathcal{B}^0$ with a non-empty support ($\operatorname{supp} \beta \neq \emptyset$). At each subsequent step $\ell+1$, the set $\mathcal{H}^{\ell+1}$ is formed by combining two components: $\mathcal{H}_A^{\ell+1}$, which retains functions from the previous level $\mathcal{H}^{\ell}$ whose support is not contained within the domain $\Omega^{\ell+1}$, and $\mathcal{H}_B^{\ell+1}$, which incorporates new functions from a refined basis $\mathcal{B}^{\ell+1}$ whose support is fully contained within $\Omega^{\ell+1}$. Finally, the ultimate set $\mathcal{H}$ is simply the result obtained at the final iteration step $N-1$.

2.2 DATA STRUCTURES FOR ANISOTROPIC MESH REFINEMENT

Efficient anisotropic mesh refinement requires data structures that can represent hierarchical spatial decompositions while accommodating refinements that differ in both direction and scale. Unlike isotropic refinement, where elements are subdivided uniformly, anisotropic refinement allows local modifications along selected dimensions, providing greater flexibility and reducing computational costs in applications where the solution exhibits directional features or localized variations.

Hierarchical data structures play a central role in managing these adaptive discretizations. They provide an efficient representation of the mesh topology, support localized refinement and coarsening operations, and facilitate neighborhood queries, traversal, and data storage. The choice

of the underlying data structure has a direct impact on the efficiency of refinement algorithms and on the complexity of the associated numerical methods.

The following subsections present three representative approaches. First, k-d trees[4] are introduced as binary space-partitioning structures that enable flexible directional refinements by recursively splitting the domain along a single coordinate axis. Next, hyperoctrees [16, 27] are hierarchical decomposition structures in which the computational domain is recursively subdivided into 2^d congruent hypercubes at each level, providing a uniform and structured refinement strategy in arbitrary dimensions. Finally, omnitrees [33] are presented as a more general hierarchical framework extending hyperoctrees, in which refinement is not required to occur simultaneously in all coordinate directions but can be selectively applied along chosen axes, thereby enabling a more flexible representation of anisotropic refinements while retaining an efficient tree-based organization. Together, these data structures provide the conceptual background for the lattice hierarchical B-spline construction investigated in the remainder of this thesis.

2.2.1 k-*d trees*

First designed by Bentley (1975) for associative searching [4], the k-d tree is a binary space-partitioning data structure initially popularized through point cloud processing and nearest-neighbor queries. Its specialized variant, the region-based k-d tree or adaptive binary tree [37], has become essential in numerical simulations and mesh generation. By recursively dividing a d-dimensional domain $\Omega \in \mathbb{R}^d$ into a binary hierarchy, it enables structured, localized adaptive mesh refinement.

The core principle of a k-d tree is the recursive bisection of the computational domain along axis-aligned hyperplanes. The root of the tree represents the entire macroscopic domain (or a base-level macro-element) Ω_0. When a local refinement is triggered, a node representing a subdomain in Ω is partitioned into two nodes such that the two resulting subdomains exactly reconstruct the original parent, ensuring a complete partition of the space with neither gaps nor overlapping regions. A simple example of this recursive subdivision process and its corresponding binary tree representation is illustrated in Figure 2.7.

Unlike isotropic subdivision schemes, which simultaneously split all coordinate directions, a k-d tree splits the domain along a single, chosen coordinate axis x_i (where $i \in \{1, \dots, d\}$) at a specific split value v. The left

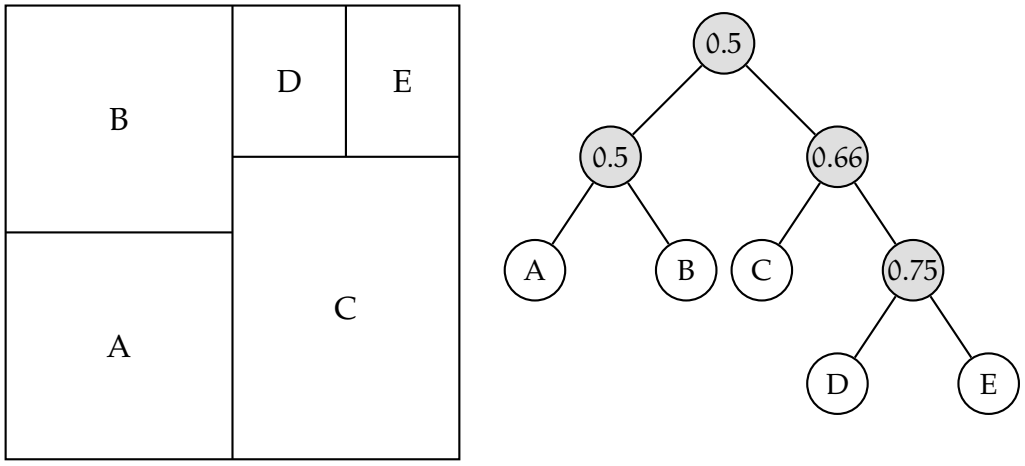


Figure 2.7: Mesh subdivision and its corresponding binary 2-d tree

child inherits the sub-region where $x_i < v$, and the right child inherits the sub-region where $x_i \geqslant v$. In practical applications, all splits are performed dyadically at the midpoint.

From an implementation perspective, an internal node in a k-d tree tailored for mesh refinement typically stores the following information:

- Pointers to the left and right child nodes.
- The scalar value $v \in \mathbb{R}$ defining the position of the split.

Leaf nodes, conversely, contain no children but store pointers to the actual mesh elements.

The hierarchical organization of a k-d tree provides a natural representation of locally refined computational domains. Since each leaf is associated with a specific region of the physical space, refinement can be performed locally without requiring any modification of distant parts of the mesh. This locality is one of the main advantages of tree-based discretizations.

Another important feature of the k-d tree is that the refinement history is explicitly stored in the tree topology. Each leaf element can be traced back through its sequence of ancestors, providing direct information about the hierarchy of approximation levels. This property simplifies operations such as refinement, coarsening, and the construction of nested discretization spaces, since the relationship between coarse and fine representations is intrinsically encoded by the data structure. In fact, the efficiency of a k-d tree originates from its hierarchical decomposition of the search space: by recursively partitioning the domain into smaller axis-aligned regions, the tree avoids the need to examine all stored entities

during spatial queries. Instead, only a subset of the branches is visited, while regions that cannot contain a valid solution are discarded through geometric pruning.

The main advantage of the resulting hierarchical structure is the acceleration of spatial queries. For example, in nearest-neighbor searches, the tree is traversed by first following the branch containing the query point and subsequently exploring only those neighboring regions that may contain closer candidates. Similarly, range queries can efficiently identify all points contained within a given region by avoiding traversal of branches whose bounding domains do not intersect the query region.

The computational performance of a k-d tree depends strongly on the balance of the hierarchy and on the dimensionality of the problem. For well-balanced trees, construction requires approximately $O(N \log N)$ operations (with N the number of elements of the tree), while nearest-neighbor and range searches typically achieve logarithmic complexity in low-dimensional spaces. However, as the number of dimensions increases, the efficiency decreases due to the so-called curse of dimensionality, since a progressively larger fraction of the tree may need to be explored during queries.

For hierarchical finite element and spline-based methods, the explicit parent-child relationship provided by the k-d tree offers a convenient framework for defining nested approximation spaces. Since each refined region remains connected to its previous coarse representation, operations involving restriction, prolongation, and basis-function activation can be performed directly from the refinement hierarchy. This makes tree-based structures particularly suitable for adaptive discretizations where both geometric refinement and hierarchical basis construction are required.

The binary subdivision mechanism allows the computational domain to be refined locally according to the distribution of the solution features. Unlike isotropic refinement techniques, where all spatial directions are subdivided simultaneously, a binary tree refinement introduces a subdivision along a single coordinate direction at each refinement step. The resulting mesh properties, however, depend on the adopted refinement strategy. In particular, classical balanced k-d tree constructions, where splitting directions are selected according to predefined rules, maintain bounded aspect ratios between neighboring elements.

Consequently, the intrinsic nature of the k-d tree imposes a structural limitation when dealing with highly directional phenomena. The tree construction restricts the ability to perform multiple, consecutive subdivisions along a single spatial coordinate. If a severe anisotropic feature

requires intensive refinement along only one axis, the standard k-d tree framework becomes insufficient, as it inevitably forces unwanted subdivisions in the other directions and therefore maintain a predefined geometric ratio: to achieve pure directional adaptivity without overrefining the mesh, supplementary methods must be introduced to bypass this limitation and allow repeated, decoupled refinements along the same dimension.

2.2.2 *Hyperoctrees*

Spatial partitioning is a fundamental concept in computational geometry, numerical simulation, and computer graphics. Many algorithms operating on geometric objects require an efficient representation of the computational domain, allowing localized operations without processing the entire space. Uniform Cartesian grids provide a straightforward discretization of the domain, partitioning the d-dimensional space into equal-sized d-rectangular elements. However, they rapidly become inefficient when the features of interest are concentrated only in limited regions. In such situations, most of the mesh elements contain little or no useful information while still contributing to the computational and memory costs.

It can be seen why this approach does not fit well in a hierarchical spline environment: hierarchical spatial data structures address this limitation by adapting the discretization to the complexity of the underlying problem. Instead of subdividing the entire domain uniformly, only the regions requiring additional resolution are recursively refined. As a consequence, computational resources are concentrated where they are actually needed while coarse cells are maintained elsewhere. This adaptive strategy significantly reduces the number of elements required to represent the domain and has consequently become one of the standard approaches in adaptive mesh refinement, geometric modeling and many other applications.

Quadtrees [16] and *octrees* [26] are widely adopted hierarchical data structures that recursively partition computational domains into regular cells, directly mapping localized mesh refinement to tree topology while enabling efficient element manipulation and neighbor retrieval. As specific instances of *hyperoctrees*, quadtrees operate in two dimensions [35] and octrees in three dimensions [36]. Hyperoctrees extend this recursive subdivision principle to arbitrary spatial dimensions, offering a unified framework for dimension-independent numerical algorithms.

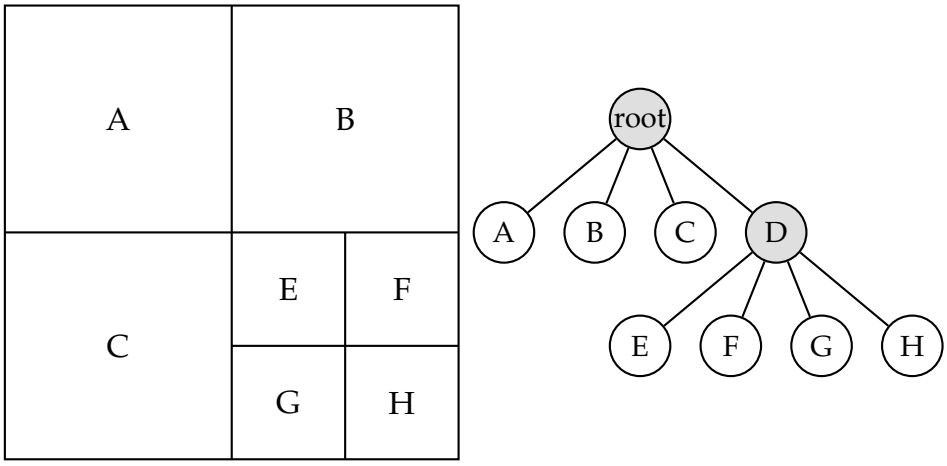


Figure 2.8: Mesh subdivision and its corresponding hyperoctree (quadtree).

Assume a bounded hypercubic computational domain $\Omega \subset \mathbb{R}^d$, where d denotes the spatial dimension. The root node of the tree represents the entire domain Ω. Each node corresponds to a hypercubic cell, which can either remain a terminal element (a leaf) or be recursively subdivided into smaller cells.

The subdivision follows a simple geometric rule. Every edge of the parent cell is bisected, producing two equal intervals along each coordinate direction. Since the subdivision is performed independently in every dimension, each refinement generates 2^d children.

Consequently,

- for $d = 2$, each square is divided into four smaller squares, producing a quadtree;
- for $d = 3$, each cube is divided into eight smaller cubes, producing an octree;
- for arbitrary dimension d, each hypercube is divided into 2^d congruent hypercubes, producing a hyperoctree.

This recursive construction is illustrated schematically in Figure 2.8.

The refinement process naturally introduces the notion of *level*. The root node belongs to level $\ell = 0$ while every refinement increases the level by one. If the initial domain has characteristic length L_0, then every refinement halves the edge length of the cells. Therefore, the characteristic size of a cell located at level ℓ is $h_\ell = \frac{L_0}{2^\ell}$.

Since all dimensions are refined simultaneously, the cell diameter decreases uniformly in every coordinate direction. The hypervolume of each cell therefore becomes $\left(\frac{L_0}{2^\ell}\right)^d$.

Thus, every refinement reduces the hypervolume by a factor of $\frac{1}{2^d}$ while generating exactly 2^d child cells.

From a topological perspective, every node of the tree possesses exactly one parent, except for the root, and either zero or 2^d children. Nodes without children are called *leaf nodes*, whereas nodes that have been subdivided are referred to as *internal nodes*. The collection of leaf nodes forms a complete partition of the computational domain, meaning that every point $\mathbf{x} \in \Omega$ belongs to one and only one leaf cell.

Unlike binary search trees or k-d trees, whose subdivision depends on the distribution of input data, hyperoctrees are fundamentally *space-driven* data structures. Their construction is determined exclusively by the geometry of the computational domain and by the refinement strategy adopted by the algorithm. Consequently, neighboring cells always remain geometrically aligned, greatly simplifying mesh generation, numerical integration, and neighborhood queries.

One of the principal advantages of hyperoctrees lies in their support for adaptive refinement. Instead of refining the entire computational domain uniformly, only those cells satisfying a prescribed refinement criterion are subdivided. Typical criteria include geometric complexity, interpolation error, local curvature, numerical residuals, or error estimators arising from finite element or isogeometric analyses. As refinement proceeds, highly resolved regions coexist with coarse cells, producing a hierarchical mesh capable of accurately representing localized features while maintaining a relatively small number of elements.

This adaptive behavior yields significant computational advantages. Since refinement is performed only where necessary, both memory consumption and computational complexity remain substantially lower than those associated with uniformly refined Cartesian grids of equivalent finest resolution. Furthermore, the hierarchical organization naturally supports recursive traversal procedures, and efficient local modifications of the computational mesh.

Nevertheless, hyperoctrees also possess intrinsic limitations. Because every subdivision simultaneously halves all coordinate directions, the generated cells remain isotropic. Consequently, directional refinement cannot be performed without introducing unnecessary subdivisions along the remaining directions. This restriction motivates the development and

usage of anisotropic hierarchical data structures, in which individual coordinate directions may be refined independently. Such structures provide greater flexibility for representing highly anisotropic solution features while preserving the hierarchical organization inherited from classical hyperoctrees.

2.2.3 *Omnitrees*

Traditional spatial structures occupy opposite ends of the subdivision spectrum: hyperoctrees enforce isotropic refinement by bisecting all coordinate axes simultaneously, producing wide but shallow trees, whereas k-d trees split along a single axis at a time, resulting in deep but narrow hierarchies. Consequently, both struggle with localized anisotropic features: hyperoctrees generate superfluous elements, while k-d trees accumulate excessive depth. To reconcile these trade-offs, omnitrees generalize both frameworks by permitting the bisection of any arbitrary subset of coordinate directions during a refinement step [33]. This flexible mechanism seamlessly accommodates both isotropic and anisotropic refinement within a single hierarchy, concurrently optimizing tree width and depth.

Formally, let $D = \{1, \dots, d\}$ denote the set of spatial dimensions. Every node v of the tree is associated with a subset $\sigma(v) \subseteq D$ which specifies the coordinate directions that are refined at that node. Rather than storing this subset explicitly, it is convenient to represent it through a binary refinement label

$$\mathbf{b}(v) = (b_1, \dots, b_d),$$

where

$$b_j = \begin{cases} 1, & j \in \sigma(v), \\ 0, & \text{otherwise.} \end{cases}$$

The number of children generated by a refinement therefore depends only on the number of refined directions: every internal node has exactly $2^{|\sigma(v)|}$ children. Classical hierarchical structures are immediately recovered as particular cases:

- if $|\sigma(v)| = d$, every dimension is bisected and the structure coincides with a hyperoctree;

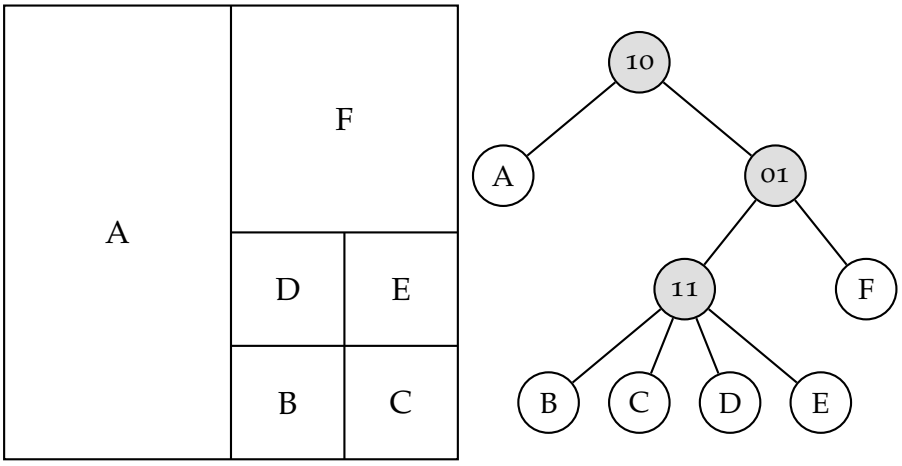


Figure 2.9: Mesh subdivision and its corresponding normalized omnitree. The leaves are the only nodes where their binary refinement label contains only zeros; they are omitted to indicate the corresponding rectangular mesh subdivision shown on the left.

- if $|\sigma(v)| = 1$, exactly one coordinate direction is refined, reproducing the behavior of a binary k-d tree;
- intermediate choices correspond to genuinely anisotropic refinements that cannot be represented directly by either structure.

An example of this result is shown in Figure 2.9.

Each node still represents an axis-aligned hyperrectangle, but unlike hyperoctrees its side lengths are no longer constrained to remain equal. The resulting cells may therefore possess different refinement levels along different coordinate directions, making the representation significantly more efficient whenever the solution exhibits directional features.

To ensure that a given spatial partition admits a unique tree representation, omnitrees are usually required to satisfy a normalization condition. Intuitively, whenever all children of a node are refined along the same direction, that refinement is promoted to the parent node instead. This process produces the shallowest possible tree for the same partition, eliminating redundant refinement levels while preserving the represented geometry.

The normalization property can be stated more formally as follows.

Definition 2.2.1 (Normalized omnitree)**.** Let there be an omnitree and let $\mathbf{b}(v)$ denote the binary refinement label associated with a node v. The

omnitree is said to be *normalized* if, for every internal node v and for every coordinate direction $j \in D$, the following implication holds:

$$\forall\, c \in \mathcal{C}(v),\ b_j(c) = 1 \quad \Longrightarrow \quad b_j(v) = 1,$$

where $\mathcal{C}(v)$ denotes the set of children of v.

In other words, if every child of a node is refined along the same coordinate direction, then that refinement is instead attributed to the parent node, avoiding redundant storage across its descendants.

This condition guarantees that every spatial partition admits a unique tree representation. Moreover, it minimizes the depth of the hierarchy by lifting common refinements as close to the root as possible, thereby eliminating redundant intermediate nodes without altering the associated subdivision of the computational domain.

The resulting flexibility has important consequences for adaptive mesh refinement. Compared to hyperoctrees, omnitrees avoid unnecessary isotropic subdivisions by refining only the directions where additional resolution is actually required. Compared to k-d trees, multiple coordinate directions can be refined simultaneously whenever appropriate, considerably reducing the overall tree depth. Consequently, omnitrees achieve a compromise between the wide branching factor of hyperoctrees and the large depth of binary trees, producing hierarchical decompositions that are both compact and well suited for highly anisotropic meshes.

This property becomes increasingly advantageous as the dimensionality grows. While a hyperoctree always generates 2^d children after each refinement, an omnitree creates only $2^{d'}$ descendants, where $d' \leqslant d$ is the number of directions that actually require refinement. For strongly anisotropic problems this substantially reduces the number of generated mesh elements and, consequently, the computational and memory costs associated with adaptive discretization.

Whenever a node is refined, only the coordinate directions contained in $\sigma(v)$ are bisected. Therefore, unlike hyperoctrees, the generated elements are not constrained to remain hypercubes. Their edge lengths may differ among coordinate directions, naturally producing anisotropic cells.

This observation motivates describing each cell by a refinement-level vector

$$\ell = (\ell_1, \ldots, \ell_d),$$

10	A	01	11	B	C	D	E	F

Figure 2.10: Flat vector representation obtained from the pre-order traversal of the tree structure in Figure 2.9.

where ℓ_j denotes the number of dyadic subdivisions performed along the j-th coordinate direction. The side length of the element along direction j is therefore proportional to $2^{-\ell_j}$.

From an implementation viewpoint, the tree structure can be encoded compactly by storing only the sequence of binary refinement labels generated during a pre-order tree traversal. In this pointerless representation, internal nodes and leaf nodes are processed sequentially: each internal node introduces its binary refinement label, followed immediately by the linearization of its subtrees.

For the structure shown in Figure 2.9, traversing the tree in pre-order [24] visits the nodes top-down from left to right, resulting in the leaf nodes (the spatial mesh elements) being ordered strictly as A,B,C,D,E,F. To linearize the tree, a sequence of each set of child indices is required. For this purpose, the standard Z-curve (Morton order) [28, 33] is commonly assumed. Due to its dimension-wise separability, the Z-curve is readily applied to omnitrees, resulting in a sequence of leaf nodes and, ultimately, of their associated spatial rectangles. This index map enables storing any omnitree-discretized function in a flat vector data structure. Combined with Morton (Z-order) indexing, this pointerless representation preserves spatial locality while reducing memory fragmentation. Furthermore, the natural nesting of the space-filling Z-curve with binary numbers allows for high cache efficiency for localized operations.

Although every node stores a binary label of length d, introducing a modest storage overhead compared with octrees, this additional cost is generally outweighed by the reduction in the number of generated cells. In highly anisotropic problems, refining only the relevant coordinate directions produces substantially smaller discretizations for the same approximation accuracy.

Omnitrees provide a natural generalization of both hyperoctrees and k-d trees. They retain the hierarchical organization and efficient traversal of classical tree structures while introducing the flexibility required for anisotropic adaptive mesh refinement, making them particularly attractive for high-dimensional numerical simulations.

3

PARTIALLY NESTED HIERARCHICAL B-SPLINE SPACES

Classical hierarchical spline constructions such as the HB-splines discussed in the previous chapter traditionally rely on a rigidly ordered sequence of isotropic refinement levels. Within this framework, the corresponding spaces and subdomains are strictly nested according to a chain of inclusions of the form

$$V^0 \subset V^1 \subset \cdots \subset V^{N-1}$$

and

$$\Omega^0 \supseteq \Omega^1 \supseteq \cdots \supseteq \Omega^n,$$

where Ω^0 represents the entire domain. Although effective, this hierarchical approach significantly restricts local refinement flexibility, since it prevents the application of independent anisotropic splitting strategies along different coordinate directions.

In this chapter, a framework for adaptive spline refinement based on lattice hierarchical B-splines (LHB-splines) is presented. Adaptive spline discretization is essential for efficient numerical approximation and isogeometric analysis (IGA) [10, 11], since local mesh refinement can resolve localized features such as sharp gradients, boundary layers, and complex geometries without unnecessary over-refinement across the entire parametric domain. Multivariate anisotropic refinement often requires local splitting along selected coordinate directions. This requirement leads to partially nested spline spaces organized in a lattice rather than in a totally ordered chain.

The remainder of this chapter is organized as follows. The mathematical framework of lattice hierarchical spaces, subdomains, mesh configurations, and LHB-splines is introduced first. The compatibility criterion,

linear independence result, and an adaptive mesh-refinement procedure are then presented. Finally, the main properties of the construction are summarized. The presentation is based on the LHB-spline framework introduced in [6].

3.1 LATTICE SPLINES ON ADAPTIVE MESHES

The discussion below concentrates on the structural change that distinguishes LHB-splines from standard HB-splines. The essential difference is not the definition of an individual tensor-product B-spline, but the organization of the refinement levels from which active functions may be selected.

3.1.1 *From isotropic to anisotropic hierarchical refinement*

In a conventional d-variate HB-spline hierarchy, a refinement step refines every coordinate direction simultaneously. The corresponding spline spaces are nested, and the characteristic element size is reduced in all directions at every step. Such isotropic refinement is appropriate near point-like singularities or features without a preferred orientation. It is inefficient, however, when the solution varies rapidly in only one or a few directions. Consider, for instance, a narrow layer or oscillatory feature whose principal direction of rapid variation is parallel to the second axis. Accurate resolution requires small elements in that direction, whereas further subdivision along the other axis contributes few useful approximation modes. An isotropic hierarchical step nevertheless halves the element size in both directions. Figure 3.1 provides an example of this, displaying a function that oscillates rapidly along one coordinate while remaining smooth along the other. In three dimensions the disparity is even more pronounced: resolving a thin surface layer may require refinement only in the direction normal to the surface, while isotropic hypertree refinement generates additional degrees of freedom in both directions.

Anisotropic refinement addresses this mismatch by permitting the resolution to be updated one direction at a time. Thus, in two dimensions, the levels $(1,0)$ and $(0,1)$ represent the number of refinement steps in the first and second coordinate direction, respectively. Neither resolution is uniformly finer than the other. Both are nevertheless descendants of $(0,0)$, and both possess the common finer level $(1,1)$. This branching and

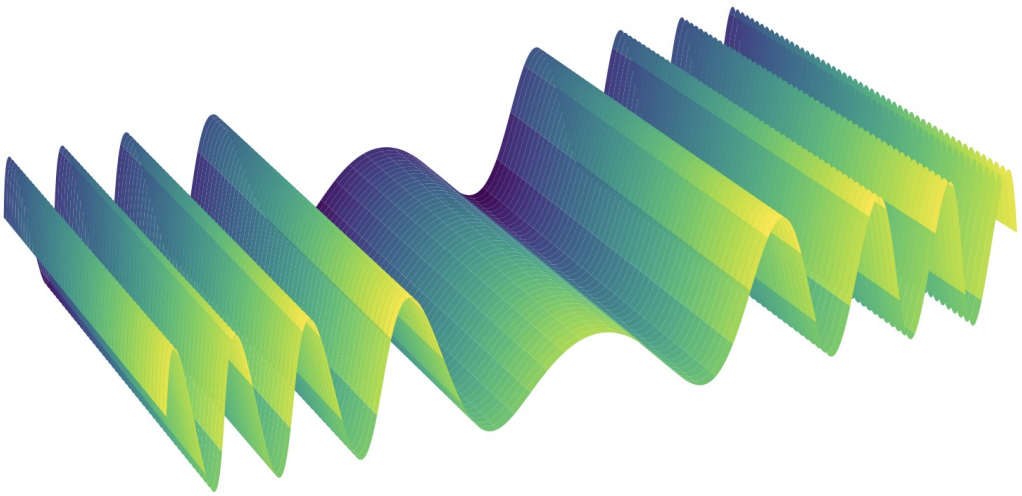


Figure 3.1: $f(x,y) = \frac{x}{20} - \frac{\sin(y^2)}{10}$

recombination cannot be represented by a single chain; it is naturally described by a lattice.

This observation also clarifies the relation with the tree structures discussed previously. A quadtree or, more generally, a d-dimensional hypertree performs a simultaneous split in all coordinate directions. A k-d tree instead records a directional split and is therefore closer to the geometric operation required by anisotropic refinement.

3.1.2 *Lattice hierarchy*

Let $d \geqslant 2$ be the dimension of the parameter domain and let

$$\mathbf{k} = (k_1, \dots, k_d), \qquad k_i \in \mathbb{N}_0, \qquad \forall i = 1 \dots d$$

denote a refinement level, which indicates the number of refinement steps applied along each coordinate direction. For two levels $\mathbf{k}, \boldsymbol{\ell} \in \mathbb{N}_0^d$,

$$\mathbf{k} \preceq \boldsymbol{\ell} \quad \Longleftrightarrow \quad k_i \leqslant \ell_i \quad \forall i = 1, \dots, d.$$

Strict precedence, denoted by $\mathbf{k} \prec \boldsymbol{\ell}$, additionally requires $\mathbf{k} \neq \boldsymbol{\ell}$. The levels are *comparable* if either $\mathbf{k} \preceq \boldsymbol{\ell}$ or $\boldsymbol{\ell} \preceq \mathbf{k}$. Otherwise they are *incomparable*, denoted by $\mathbf{k} \parallel \boldsymbol{\ell}$.

If $\mathbf{k} \preceq \boldsymbol{\ell}$, a mesh at level $\boldsymbol{\ell}$ is at least as fine as a mesh at level $\mathbf{k}$ in every direction. If $\mathbf{k} \parallel \boldsymbol{\ell}$, both $\mathbf{k}$ and $\boldsymbol{\ell}$ are finer in at least one direction and coarser in another. For example, $(1,0) \parallel (0,2)$, whereas $(1,0) \preceq (1,2)$.

Let $\mathbf{e}_i$ be the i-th Cartesian unit vector. The immediate successors of level $\mathbf{k}$ are

$$\text{children}(\mathbf{k}) := \{\mathbf{k} + \mathbf{e}_i : i = 1, \dots, d\}.$$

Therefore, the number of children is equal to the number of dimensions. The root is $\mathbf{0} = (0, \dots, 0)$. For a finite construction, only levels with $|\mathbf{k}| \leqslant f_{\max}$ are retained, where

$$|\mathbf{k}| := \sum_{i=1}^{d} k_i$$

is the floor index. Levels on the same floor contain the same total number of directional refinement steps, although their resolutions may differ.

The componentwise minimum and maximum are defined as

$$\mathbf{k} \wedge \boldsymbol{\ell} := \big(\min(k_1, \ell_1), \dots, \min(k_d, \ell_d)\big),$$
$$\mathbf{k} \vee \boldsymbol{\ell} := \big(\max(k_1, \ell_1), \dots, \max(k_d, \ell_d)\big).$$

The minimum yields the closest common predecessor, while the maximum yields the closest common successor. Thus, $(2, 0)$ and $(0, 1)$ have the closest common predecessor $(0, 0)$ and the closest common successor $(2, 1)$.

For every level $\mathbf{k}$, let $V^{\mathbf{k}}$ be a d-variate tensor-product spline space on $\Omega = [0, 1]^d$, spanned by the B-spline basis $\mathcal{B}^{\mathbf{k}}$. A fixed degree m and continuity C^{m-1} at interior knots are assumed. The initial mesh $\mathcal{G}^{\mathbf{0}}$ has open knot vectors, and $\mathcal{G}^{\mathbf{k}+\mathbf{e}_i}$ is obtained by uniform dyadic refinement of $\mathcal{G}^{\mathbf{k}}$ only in direction i.

Directional knot insertion gives

$$\mathbf{k} \preceq \boldsymbol{\ell} \quad \Longleftrightarrow \quad V^{\mathbf{k}} \subseteq V^{\boldsymbol{\ell}}.$$

The lattice hierarchy is therefore partially nested. Incomparable levels generally define non-nested spaces: neither $V^{(1,0)} \subseteq V^{(0,1)}$ nor the reverse inclusion holds, although both spaces are contained in $V^{(1,1)}$. As detailed in Figure 3.2, the refinement level lattice establishes the hierarchy together with its governing rules; given two levels $\mathbf{k}$ and $\boldsymbol{\ell}$, with $|\mathbf{k}| \leqslant |\boldsymbol{\ell}|$, without loss of generality, they are comparable if there is a downward path connecting them, that is $\mathbf{k} \wedge \boldsymbol{\ell} = \mathbf{k}$ and $\mathbf{k} \vee \boldsymbol{\ell} = \boldsymbol{\ell}$.

A closed, grid-aligned subdomain $\Omega^{\mathbf{k}} \subseteq \Omega^{\mathbf{0}}$ is assigned to each level. The ordering is reversed with respect to the spline spaces:

$$\boldsymbol{\ell} \preceq \mathbf{k} \quad \Longleftrightarrow \quad \Omega^{\mathbf{k}} \subseteq \Omega^{\boldsymbol{\ell}}.$$

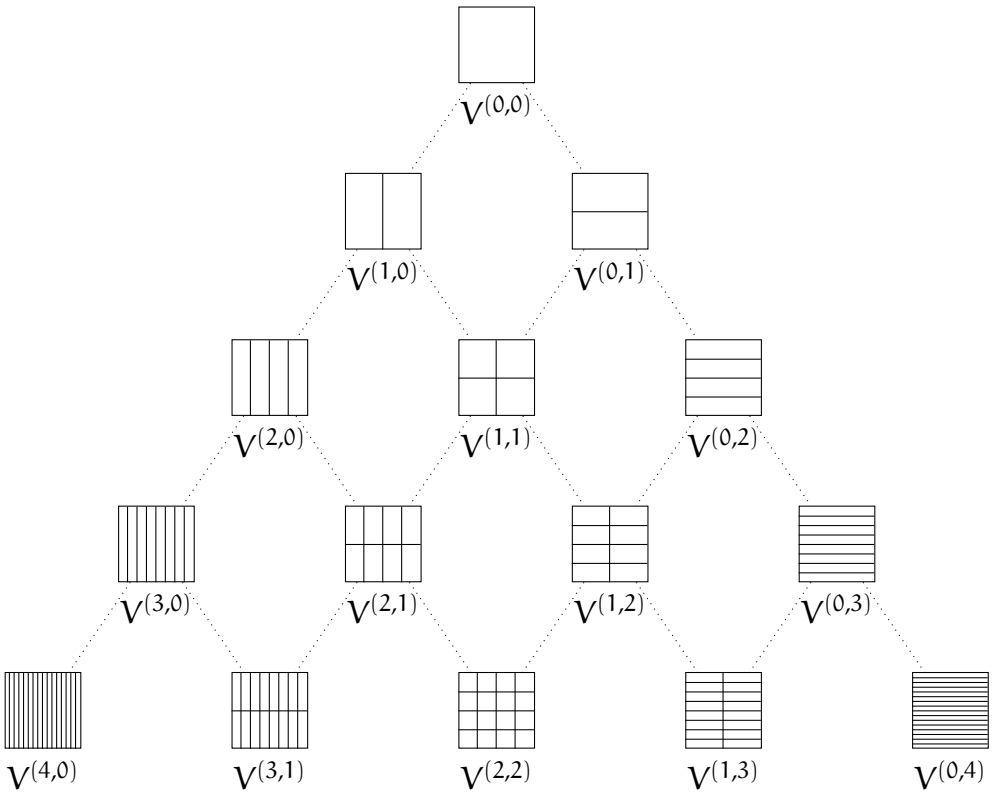


Figure 3.2: Lattice hierarchy with $f_{max} = 4$.

For $|\mathbf{k}| < f_{max}$, the local partition is

$$\pi^{\mathbf{k}} := \Omega^{\mathbf{k}} \setminus \operatorname{Int}\left(\bigcup_{\ell \in \text{children}(\mathbf{k})} \Omega^{\ell} \right).$$

On the last floor, $\pi^{\mathbf{k}} := \Omega^{\mathbf{k}}$.

The interiors of non-empty partitions are mutually disjoint and cover Ω. The active element levels are

$$\mathcal{I} := \{\mathbf{k} : \pi^{\mathbf{k}} \neq \emptyset\}.$$

The distinction between $\Omega^{\mathbf{k}}$ and $\pi^{\mathbf{k}}$ is essential. The former contains every region whose resolution succeeds $\mathbf{k}$; the latter contains only the region represented actively at level $\mathbf{k}$.

3.1.3 *Lattice hierarchical B-splines*

The active elements of level $\mathbf{k}$ are

$$\mathcal{Q}^{\mathbf{k}} := \left\{ Q \in \mathcal{G}^{\mathbf{k}} : Q \subseteq \Omega^{\mathbf{k}} \wedge Q \not\subseteq \Omega^{\ell},\ \forall \ell \in \text{children}(\mathbf{k}) \right\}. \tag{3.1}$$

The lattice hierarchical mesh is

$$\mathcal{Q} := \bigcup_{\mathbf{k} \in \mathcal{J}} \mathcal{Q}^{\mathbf{k}}. \tag{3.2}$$

It is equivalent to the chosen subdomain hierarchy. Refinement can therefore be expressed by updating the subdomains, while integration and assembly operate on active leaves of an adaptive tree.

The selection rule extends the support-based definition of HB-splines. A B-spline is eligible if its support lies in the subdomain assigned to its level, and remains active unless one immediate child subdomain covers the complete support.

Definition 3.1.1 (Lattice hierarchical B-spline)**.** Given the lattice hierarchical mesh $\mathcal{Q}$, the corresponding lattice hierarchical B-spline (LHB-spline) $\mathcal{L}(\mathcal{Q})$ consists of the collection of tensor-product B-splines selected across all refinement levels according to the following:

$$\mathcal{L}(\mathcal{Q}) := \bigcup_{|\mathbf{k}| \leqslant \mathsf{f}_{\max}} \left\{ B^{\mathbf{k}} \in \mathcal{B}^{\mathbf{k}} : \operatorname{supp}(B^{\mathbf{k}}) \subseteq \Omega^{\mathbf{k}},\ \operatorname{supp}(B^{\mathbf{k}}) \not\subseteq \Omega^{\ell}\ \forall \ell \in \text{children}(\mathbf{k}) \right\}.$$

Furthermore,

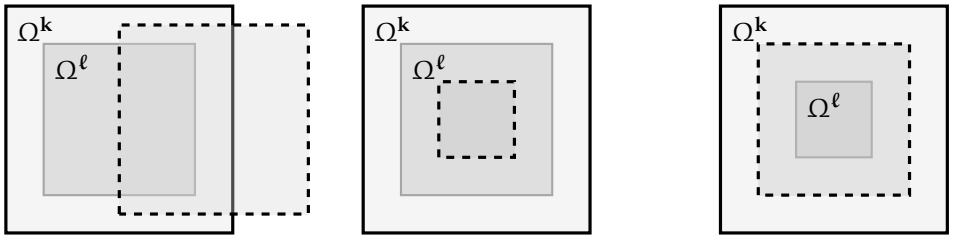
$$\mathcal{L}^{\mathbf{k}}(\mathcal{Q}) := \mathcal{L}(\mathcal{Q}) \cap \mathcal{B}^{\mathbf{k}}, \qquad \mathcal{J} := \{\mathbf{k} : \mathcal{L}^{\mathbf{k}}(\mathcal{Q}) \neq \emptyset\}.$$

The first condition in 3.1.1 prevents activation outside the region where the selected resolution is valid. The second deactivates a function if its support lies entirely in the subdomain of a child level. Only immediate children need to be tested: a function whose support lies within a deeper descendant is automatically enclosed by its direct child.

Three geometric cases can be distinguished for functions:

1. A function whose support extends outside $\Omega^{\mathbf{k}}$ is rejected, thus not activated.

2. A support lying in $\Omega^{\mathbf{k}}$ and completely covered by one child subdomain is deactivated.



(a) Inactive (b) Deactivated (c) Active

Figure 3.3: Geometric interpretation of the LHB-spline activation rule as described in 3.

3. A support lying in Ω^k but not completely covered by any single child remains active.

These three possible configurations are shown in Figure 3.3, from left to right. The dashed rectangles in the figures denote a spline support at level k, and the different gray regions represent distinct subdomains, in particular $\Omega^k \supset \Omega^\ell$. In the first case 3.3a, the support of the considered function is not entirely contained in Ω^k, therefore it is not eligible for activation. In the second case 3.3b, the support is contained in Ω^k and is completely covered by the subdomain Ω^k; consequently, the function is deactivated and replaced by functions from the corresponding finer level. In the third case 3.3c, the support is contained in Ω^k, but no child subdomain covers it completely. The function therefore remains active.

The third case may occur because the support intersects π^k, as in an ordinary HB hierarchy, or because different pieces are completely covered by their children, which are incomparable descendants. The latter possibility is specific to LHB-splines and gives rise to ghost functions.

If an active cell Q^ℓ is contained in the support of an active B^k, partial nestedness implies $k \preceq \ell$. An active function may influence cells at its own or a succeeding resolution, but not cells at coarser or incomparable levels.

Definition 3.1.2 (Ghost function)**.** An active $B^k \in \mathcal{L}^k(\Omega)$ is a ghost function if every active cell $Q^\ell \subseteq \operatorname{supp}(\mathcal{B}^k)$ satisfies $k \prec \ell$.

Equivalently, a ghost support contains no active cell of its own level and does not intersect the interior of π^k. The function is selected by the subdomain test, all active cells within its support belong to finer levels, and since no active cell of the same level lies within the support of a ghost function, the function is not directly identifiable from the active mesh. Moreover, its contribution may be reproduced by active functions



(a) Active non-ghost function (b) Active ghost function

Figure 3.4: Geometric configuration of active LHB-spline basis functions. The support (dashed line) contains an active cell at its own level $\mathbf{k}$ in (a), whereas in (b) it is entirely covered by finer cells of descendant levels, making it a ghost function.

belonging to finer levels, potentially causing non-uniqueness of the spline coefficients and linear dependence.

The geometric distinction between the possible configurations of active functions is illustrated in Figure 3.4, where:

- In the left configuration (a), the support intersects an active mesh element of level $\mathbf{k}$, so the function is a non-ghost function.
- In the right configuration (b), the support contains no active mesh elements of level $\mathbf{k}$ and is entirely covered by cells of the incomparable descendant levels $\mathbf{k}+\boldsymbol{e}_0$ and $\mathbf{k}+\boldsymbol{e}_1$, making it a ghost function.

Ghost functions arise when the support of a function intersects partitions associated with incomparable descendant levels. To illustrate this concept, consider a domain $\Omega^{(1,1)}$ split into two subregions governed by incomparable refinement levels: one containing active cells of level $(3,1)$, obtained via three refinements in the first coordinate direction and one in the second, and another containing active cells of level $(1,3)$, refined analogously. Their closest common predecessor level is $(1,1)$. Now consider a coarse function $\beta \in \mathcal{B}^{(1,1)}$ defined over $\Omega^{(1,1)}$ whose support overlaps both subregions. Although this support is entirely covered by finer active cells, neither of the immediate child subdomains, $\Omega^{(2,1)}$ or $\Omega^{(1,2)}$, completely contains it. Because no single finer level can fully represent β, the function is activated at level $(1,1)$ to preserve the completeness of the basis, even though every active cell within its support belongs to a level strictly succeeding $(1,1)$. Hence, β is a ghost function.

Linear dependence destroys uniqueness of spline coefficients and makes mass or stiffness matrices [20] singular unless constraints or ba-

sis reduction are introduced. This ambiguity in the degrees of freedom directly compromises the reliability of error estimation and mesh coarsening. Consequently, enforcing topological compatibility to prevent linear dependencies would be preferable than handling singular systems after assembly.

3.2 COMPATIBLE MESHES AND LINEAR INDEPENDENCE

Compatibility constrains local combinations of element levels without eliminating anisotropic refinement. It requires all strictly finer active elements within any B-spline support to share at least one first refinement direction.

Definition 3.2.1 (Compatible Mesh)**.** A mesh $\mathcal{Q}$ is compatible if, for every $B^{\mathbf{k}} \in \mathcal{B}^{\mathbf{k}}$, there exists $\mathbf{k}^+ \in \text{children}(\mathbf{k})$ such that every active cell $Q^{\boldsymbol{\ell}} \subseteq \text{supp}(B^{\mathbf{k}})$ satisfies

$$\mathbf{k} \prec \boldsymbol{\ell} \quad \Longrightarrow \quad \mathbf{k}^+ \preceq \boldsymbol{\ell}.$$

The child $\mathbf{k}^+$ does not have to be unique nor be an active cell level. It records a shared directional refinement. Equivalently, for two active cell levels $\boldsymbol{\ell}_1, \boldsymbol{\ell}_2 \succ \mathbf{k}$ inside the same support,

$$\mathbf{k} \prec \mathbf{k}^+ (\preceq \boldsymbol{\ell}_1 \wedge \boldsymbol{\ell}_2).$$

Hence, $\mathbf{k}$ cannot be their closest common predecessor.

As a concrete two-dimensional example, consider a B-spline $\beta \in \mathcal{B}^{(1,1)}$ whose support contains active cells with levels

$$\boldsymbol{\ell}_1 = (3,2) \qquad \text{and} \qquad \boldsymbol{\ell}_2 = (2,4).$$

These levels are incomparable, since $\boldsymbol{\ell}_1$ is finer in the first coordinate direction, whereas $\boldsymbol{\ell}_2$ is finer in the second one. Nevertheless, their closest common predecessor is

$$\boldsymbol{\ell}_1 \wedge \boldsymbol{\ell}_2 = (2,2),$$

which strictly succeeds $\mathbf{k} = (1,1)$. In particular, either child

$$\mathbf{k}^+ = (2,1) \qquad \text{or} \qquad \mathbf{k}^+ = (1,2)$$

may be selected, since both satisfy

$$\mathbf{k} \prec \mathbf{k}^+ \preceq (2,2) \preceq \boldsymbol{\ell}_1$$
$$\mathbf{k} \prec \mathbf{k}^+ \preceq (2,2) \preceq \boldsymbol{\ell}_2$$

Therefore, all finer active cells within the considered support share at least one refinement step beyond level $(1,1)$, and the local configuration is compatible. Notice that anisotropy is preserved: compatibility does not require $\boldsymbol{\ell}_1$ and $\boldsymbol{\ell}_2$ to coincide or even to be comparable.

For comparison, if the same support contained cells of levels $(3,1)$ and $(1,4)$, then

$$(3,1) \wedge (1,4) = (1,1) = \mathbf{k}.$$

In this case, no child of $(1,1)$ precedes both active levels. The two refinement branches separate immediately above $\mathbf{k}$, and the resulting local configuration is therefore incompatible.

3.2.1 *Elimination of ghost functions*

Assume that an active $B^{\mathbf{k}}$ were a ghost. All active cells in its support would have levels strictly succeeding $\mathbf{k}$. Compatibility would provide a child $\mathbf{k}^+$ preceding every such level. The entire support would then lie in $\Omega^{\mathbf{k}^+}$, contradicting the activation rule. Consequently,

$$\mathcal{Q} \text{ compatible} \quad \Longrightarrow \quad \operatorname{supp}(B^{\mathbf{k}}) \cap \pi^{\mathbf{k}} \neq \emptyset \quad \forall B^{\mathbf{k}} \in \mathcal{L}(\mathcal{Q}).$$

Thus $\mathcal{J} \subseteq \mathcal{I}$. Compatibility is sufficient but not necessary: some incompatible meshes may still produce independent sets, but the local condition gives an a priori guarantee.

If $\mathcal{Q}$ is compatible, the functions in $\mathcal{L}(\mathcal{Q})$ are linearly independent [6]. It guarantees uniqueness of discrete representations and nonsingularity of matrices. It also enables stable basis evaluation, meaningful dimension counts, and reliable coarsening.

The neighborhood relevant to compatibility is not limited to cells sharing a face with the marked element. Since a B-spline support generally spans several mesh cells, a more distant active cell may still interact with the marked element through a basis function that is non-zero on both of them. The *support extension* $S(Q^{\boldsymbol{\ell}})$ collects the supports of all level-$\boldsymbol{\ell}$ B-splines that are non-zero on $Q^{\boldsymbol{\ell}}$. It therefore identifies the complete region in which the refinement of the marked cell may create incompatible combinations of active levels.

For $Q^{\boldsymbol{\ell}} \in \mathcal{G}^{\boldsymbol{\ell}}$,

$$S(Q^{\boldsymbol{\ell}}) := \bigcup_{\substack{B^{\boldsymbol{\ell}} \in \mathcal{B}^{\boldsymbol{\ell}} \\ \operatorname{supp}(B^{\boldsymbol{\ell}}) \cap Q^{\boldsymbol{\ell}} \neq \emptyset}} \operatorname{supp}(B^{\boldsymbol{\ell}}). \tag{3.3}$$

3.2.2 *Comparable marking*

To ensure algebraic stability and prevent mesh inconsistencies during adaptive refinement, the LHB-splines rely on *comparable marking*, a conservative strategy designed to evaluate whether a desired jump in refinement levels violates compatibility with neighboring cells in the support extension. Within this procedure, suppose there is a cell Q^{ℓ_M} that has already been marked for refinement, and a requested target level $\ell_T \succ \ell_M$. This is the operational starting point of the algorithm: determine whether the desired jump in refinement levels is safe for the mesh or if it risks breaking compatibility with the neighboring cells in the support extension. To do this, the algorithm must evaluate all possible future levels and identify the optimal one. Instead of blindly enforcing the target, the algorithm analyzes the local neighborhood to identify the minimal, safest level that satisfies both the refinement demand and the global mesh compatibility rules. Then, the active levels in its support extension are collected:

$$\mathcal{N}_S := \{\mathbf{k} : Q^{\mathbf{k}} \in \mathcal{Q},\ Q^{\mathbf{k}} \subseteq S(Q^{\ell_M})\}.$$

Starting from $\ell_{MT} = \ell_T$, every incomparable $\mathbf{k} \in \mathcal{N}_S$ produces

$$\ell_{MT} \leftarrow \ell_{MT} \vee \mathbf{k}.$$

The incomparable subset is recomputed until none remains. Each neighborhood level is processed at most once, so the procedure terminates. The output is comparable with all levels in the support extension and is the admissible target on the smallest floor.

Let $\mathcal{N}_S$ denote the set of levels of all active cells contained in the support extension of the marked cell, and let ℓ_T be the target level initially requested for its refinement. A candidate level $\mathbf{j}$ is considered admissible if it succeeds the requested target level and is comparable with every level encountered in the support extension. The set of all such admissible levels is therefore defined as

$$\mathcal{A}(\mathcal{N}_S, \ell_T) := \left\{\mathbf{j} \in \mathbb{N}_0^d : \mathbf{j} \succeq \ell_T \text{ and } \mathbf{j} \text{ is comparable with every } \mathbf{k} \in \mathcal{N}_S\right\}.$$

Among these candidates, the comparable-marking procedure selects the level requiring the smallest total number of directional refinement steps. Since the floor index of a level $\mathbf{j} = (j_1, \ldots, j_d)$ is defined by

$$|\mathbf{j}| = \sum_{r=1}^{d} j_r,$$

the modified target level is given by

$$\ell_{MT} = \operatorname*{arg\,min}_{\mathbf{j} \in \mathcal{A}(\mathcal{N}_S, \ell_T)} |\mathbf{j}|.$$

Thus, ℓ_{MT} is the admissible level on the lowest possible floor: it preserves comparability with the neighboring active levels while introducing the minimum additional refinement required by compatibility. In practice, among all possible future refinement levels that are higher than the target and resolve neighbor conflicts, the lowest and most conservative one among them is strictly selected.

The activation of an LHB-spline must be determined from its entire support. Mesh compatibility cannot be verified by inspecting only cells that share an edge, or a face in higher dimensions. A B-spline of degree m generally extends over several cells in each coordinate direction. Consequently, two cells that are not adjacent may still belong to the support of the same basis function and may therefore contribute to an incompatible combination of levels. The support extension of a marked cell identifies all cells that can interact with it through such common B-spline supports and thus provides the appropriate neighborhood for compatibility checks.

These observations highlight the two central principles of the LHB-spline construction: anisotropic refinement must be described through the full multi-index of each level, and both basis selection and mesh compatibility must be assessed over complete B-spline supports. Together with compatible marking, these principles make it possible to retain the flexibility of directional refinement while preserving the linear independence required for a reliable spline discretization.

4

ALGORITHMS AND DATA STRUCTURES FOR LHB-SPLINES

The theoretical constructions introduced in Chapters 2 and 3 become useful in an adaptive approximation procedure only after two complementary representations have been connected. The first representation is geometric: it stores a dyadic partition of the parameter domain and permits local, direction-dependent refinement. The second is functional: it associates the partition with the partially nested tensor-product spaces from which the active LHB-splines are selected. This chapter describes that connection and the design decisions made in its implementation.

The discussion concentrates on algorithms and data flow, supported by selected Python excerpts that expose the main implementation choices. Section 4.1 describes the representation and refinement logic of the omnitree implementation, based on DyAda [32], and of the cyclic geometric k-d tree used as a reference structure. After that, it presents how an omnitree mesh is converted into a lattice hierarchy of dyadic spline spaces and how the active LHB-splines are identified from their complete supports are explained. In Section 4.2, the two tree structures are compared using four distinct cases: an illustrative mesh, a strongly anisotropic construction, a mesh derived from function approximation, and randomly generated examples in two, three, and four dimensions. The comparison separates the unavoidable cost of representing the same leaves from the additional geometric refinement imposed by the cyclic splitting rule of the k-d tree.

The methodology presented in this work is implemented in a open-source Python library developed by the author of this thesis at [8].

4.1 PYTHON IMPLEMENTATION OF k-D TREES AND OMNITREES

The purpose of this section is to describe how the two structures are represented in the developed Python code, how refinement requests are

applied, and how comparable statistics are obtained. The excerpts are intentionally limited to the statements that expose the algorithmic choices; validation, plotting, and file-management instructions are omitted.

The implementation works on a tensor-product domain discretized by an initial regular mesh, where the number of intervals along each spatial direction is a power of two. The active mesh is represented through its leaf boxes, each characterized by dyadic bounds and a directional level vector. The omnitree implementation of descriptor manipulation–the binary string indicating the linearized representation of a specific omnitree–and Morton ordering are fully handled by DyAda, while the k-d trees are constructed explicitly as a recursive collection of Python nodes.

4.1.1 *Construction and refinement of the DyAda discretization*

An omnitree is initialized from its dimension, its Morton linearization, and a compact binary refinement descriptor. Listing 4.1 shows the complete initialization.

```
discretization = dyada.Discretization(
    dyada.MortonOrderLinearization(),
    dyada.RefinementDescriptor.from_binary(
        num_dimensions,
        initial_mesh,
    ),
)
```

Listing 4.1: Initialization of an omnitree discretization from a binary descriptor.

This representation has two useful consequences for the remainder of the implementation. First, the descriptor can be passed directly between mesh refinement and basis construction without translating it into a second tree class. Second, the order used to enumerate leaves is fixed by the Morton linearization, so arrays of element values can follow the same ordering as the mesh.

Refinements are collected in a planned operation rather than being applied one by one to an already changing descriptor. The wrapper used in the mesh-refinement script is shown in Listing 4.2. Each pair identifies the target recognized by DyAda and the binary direction mask to be applied there.

```
def apply_refinements(discretization, *refinements):
    plan = dyada.PlannedAdaptiveRefinement(discretization)

```

```
    for level, direction in refinements:
        plan.plan_refinement(level, direction)

    discretization, index_mapping = plan.apply_refinements(
        track_mapping="boxes"
    )
    return discretization, index_mapping
```

Listing 4.2: Application of a batch of planned omnitree refinements.

The call that applies the plan returns both the refined discretization and an old-to-new box mapping. The mapping is not needed when only the final geometry is plotted, but it is important in an adaptive solver because element data must be transferred after the descriptor has been normalized and the Morton order may have changed. Returning the mapping at the same time as the mesh update prevents values from being associated with stale leaf positions.

The normalization process itself is performed inside DyAda: the local code specifies what is to be refined, the library resolves the corresponding normalized descriptor and new leaf order. The normalization theory is described in Section 2.2.3.

The code also supports direct descriptor construction for reproducible test meshes. A hand-written bit array is useful for small examples because it fixes the complete topology, whereas planned refinements are more convenient when the mesh is produced by an adaptive criterion. Both routes create the same `Discretization` object and therefore expose the same interface to the later LHB routines.

4.1.2 *Explicit representation of the cyclic k-d tree*

The k-d tree is implemented explicitly because its structure is sufficiently small and because direct access to node depths and bounds simplifies the comparison. A node stores the geometric bounds, the depth, and the binary path used to reach it. Internal nodes additionally store the selected axis, midpoint, and two children. Listing 4.3 contains the corresponding data model.

```
@dataclass(slots=True)
class KDNode:
    bounds: tuple[tuple[float, float], ...]
    depth: int
    path: str
    split_axis: int | None = None
    split_value: float | None = None
```

```
    children: tuple["KDNode", "KDNode"] | None = None
    is_leaf: bool = True
```

Listing 4.3: Node information stored by the geometric k-d tree.

The use of slots keeps the node layout fixed and avoids an additional dynamic attribute dictionary for every node. This does not alter the tree algorithm, but it is appropriate for experiments in which the converted k-d tree may contain hundreds of thousands of nodes. The `is_leaf` flag is stored explicitly so that statistics and plotting do not have to infer it repeatedly from the children field.

The constructor receives a collection of binary refinement paths. Before recursion begins, every prefix of every requested path is inserted into an active-prefix set. This ensures that a descendant can never be requested without its ancestors. The central part of the constructor is shown in Listing 4.4.

```
active_paths = {
    path[:i]
    for path in refinement_paths
    for i in range(len(path) + 1)
}

def build_node(current_bounds, depth, path):
    node = KDNode(current_bounds, depth, path)

    if path not in active_paths:
        return node

    split_axis = depth % num_dimensions
    lower, upper = current_bounds[split_axis]
    split_value = 0.5 * (lower + upper)

    left_bounds = list(current_bounds)
    right_bounds = list(current_bounds)
    left_bounds[split_axis] = (lower, split_value)
    right_bounds[split_axis] = (split_value, upper)

    node.children = (
        build_node(tuple(left_bounds), depth + 1, path + "0"),
        build_node(tuple(right_bounds), depth + 1, path + "1"),
    )
    node.split_axis = split_axis
    node.split_value = split_value
    node.is_leaf = False
    return node
```

Listing 4.4: Recursive midpoint construction of the cyclic k-d tree.

Three implementation decisions are visible in this excerpt. The split is geometric rather than data-driven, since the midpoint is used instead of a sample median. The axis is determined by `depth % num_dimensions`, which produces the fixed cyclic order used throughout the comparison. Finally, refining a node always creates both children, so the result is a full binary tree. These choices make the baseline deterministic, but they also explain the additional cells created when the desired refinement repeatedly selects the same direction.

The path string is both a construction key and a compact description of the sequence of lower and upper half-boxes. It is not sufficient by itself to recover the geometry unless the dimension and the cyclic axis convention are also known, which is why bounds are retained in every node. Storing the bounds makes plotting and leaf conversion immediate and avoids replaying the entire path for each geometric query.

4.1.3 *Conversion used by the comparison and statistics collected*

The k-d tree examples are derived from the already constructed omnitree meshes. The leaf boxes of the omnitree are enumerated, their dyadic boundaries are recovered, and the prefix paths required to reproduce those boundaries with cyclic midpoint splits are generated. The resulting paths are then passed to the constructor in Listing 4.4. The intermediate serialized mesh descriptions are only transport representations and do not affect the comparison.

When the omnitree refinement directions already follow the cyclic k-d tree order, the conversion can preserve the leaf partition exactly. These are the fair examples of Section 4.2.4. Otherwise, the k-d tree must continue splitting until the required axis returns in the cycle. Subdivisions introduced in the intervening directions remain part of the final conforming binary mesh. The additional k-d tree leaves are therefore generated by the representation constraint, not by a second error estimator or an independent random process.

The statistics routine traverses each final tree and counts leaves, total nodes and computes the average depth. For the explicit k-d tree, these values follow directly from `KDNode.depth` and `KDNode.is_leaf`. For the omnitree, the descriptor is traversed according to its stored branching masks.

The leaf count measures the size of the active geometric partition. The internal-node count measures the topology required to organize that partition. Average leaf depth estimates the number of hierarchical

	Leaves			Total nodes			Average depth		
Case	Omnitree	k-d tree	ratio	Omnitree	k-d tree	ratio	Omnitree	k-d tree	ratio
Illustrative 2D	14	14	100.00%	23	27	85.19%	3.07	4.42	69.46%
Anisotropic 2D	8	32	25.00%	15	63	23.81%	3.00	5.00	60.00%
2D fair	501	501	100.00%	813	1001	81.22%	6.19	11.02	56.17%
3D fair	1501	1501	100.00%	2375	3001	79.14%	5.78	13.22	43.72%
4D fair	2001	2001	100.00%	3181	4001	79.51%	5.87	13.88	42.29%
2D unrestricted	2695	15700	17.17%	4019	31399	12.80%	7.24	15.96	45.36%
3D unrestricted	4241	876963	0.48%	5787	1753925	0.33%	10.22	36.82	27.76%
4D unrestricted	8067	745659	1.08%	10115	1491317	0.68%	8.95	38.19	23.44%

Table 4.1: Structural statistics for the omnitree and cyclic k-d tree representations.

steps encountered by a typical root-to-leaf traversal. Execution time is intentionally not inferred from these quantities: DyAda uses a compact descriptor, whereas the reference k-d tree uses Python objects, so a timing comparison would mix tree structure with storage implementation. The structural statistics isolate the effect of the refinement model itself.

4.2 COMPARISON

The purpose of the comparison is to measure the structural price paid when a general anisotropic dyadic partition is represented by a binary tree with a prescribed cyclic axis order. Omnitrees and k-d trees are evaluated from the same starting geometry according to the conversion described in Section 4.1.3. The results therefore compare representations rather than unrelated refinement criteria.

4.2.1 *Protocol and interpretation of the data*

For each case, the recorded quantities are the numbers of leaves and total nodes, together with the average leaf depth. Table 4.1 reports the complete data.

The random meshes are single stored realizations, not averages over an ensemble, because the generator selects both the leaf and the refinement mask randomly without fixing a seed. They are used to expose structural trends, while the deterministic anisotropic example provides the clearest causal demonstration. The term *fair* means that both structures are constrained to have the same number of leaves.

4.2.2 *Illustrative two-dimensional example*

The mesh of the first example is shown in Figure 4.1. It is simple enough that both trees can be represented graphically: it contains fourteen active rectangles with several local aspect ratios. The binary representation expands the construction into a sequence of single-axis decisions, whereas the normalized omnitree combines compatible directional decisions at common ancestors.

As reported in Table 4.1 (line 1), both structures have fourteen leaves. The omnitree uses nine internal nodes and twenty-three total nodes; the k-d tree uses thirteen internal nodes and twenty-seven total nodes. The four-node difference is entirely hierarchical overhead, since the active partition is unchanged. More importantly, the average leaf depth decreases from 4.42 to 3.07, a reduction of approximately 30.5%. This means that the improvement is not confined to one exceptional branch: it affects the typical root-to-leaf traversal.

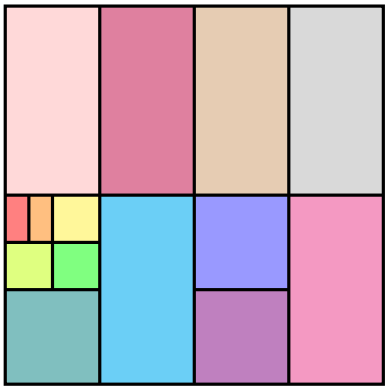
The example also clarifies why comparing only leaves can hide a benefit. Even when the k-d tree can represent the same boxes exactly, its binary topology must serialize independent directional refinements. Normalization permits the omnitree to carry them in one node and to lift common splits. The active discretization size is identical, but the route used to reach it is shorter and contains fewer intermediate boxes.

4.2.3 *Strongly anisotropic two-dimensional refinement*

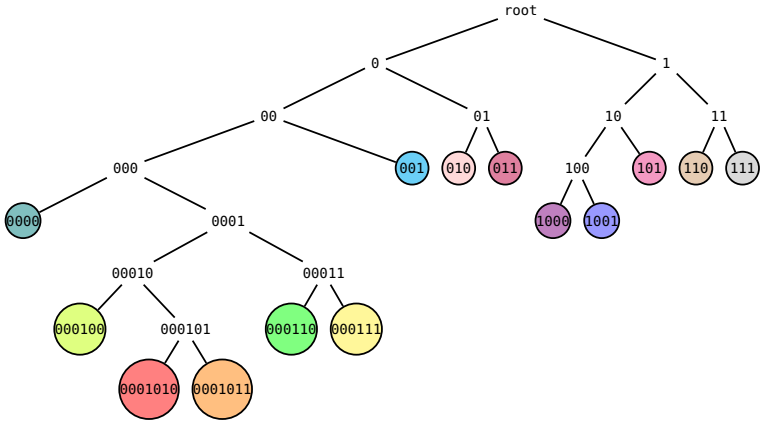
The deterministic anisotropic example isolates the principal limitation of the cyclic k-d tree. The desired mesh consists of eight vertical strips, obtained by three consecutive bisections in the first coordinate direction and no subdivision in the second. The omnitree expresses this request directly with three binary refinements along the same axis. Its final mesh is shown on the left of Figure 4.2.

In two dimensions the cyclic sequence is $x, y, x, y, \ldots$. After the first x split, the second x split cannot occur until a y split has been inserted, and the third x split requires a second y split. Five binary levels are consequently required to perform the three desired x refinements. The k-d tree therefore constructs a uniform 8×4 partition with 32 leaves, even though the target resolution varies only in x. The horizontal lines on the right of Figure 4.2 have no counterpart in the refinement request.

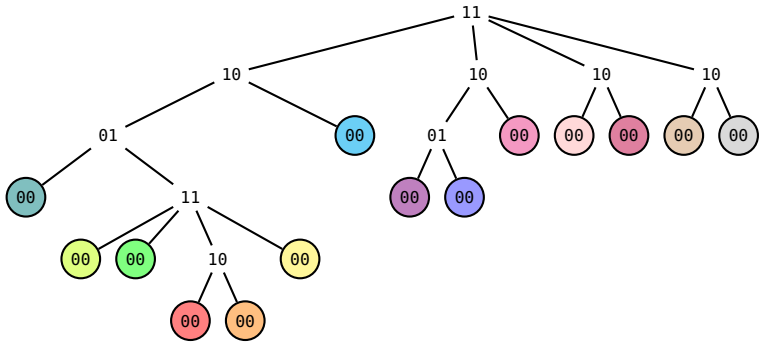
Because both trees are binary in this particular case, the internal-node counts grow in the same manner: seven internal nodes and fifteen total



(a) Colored adaptive mesh

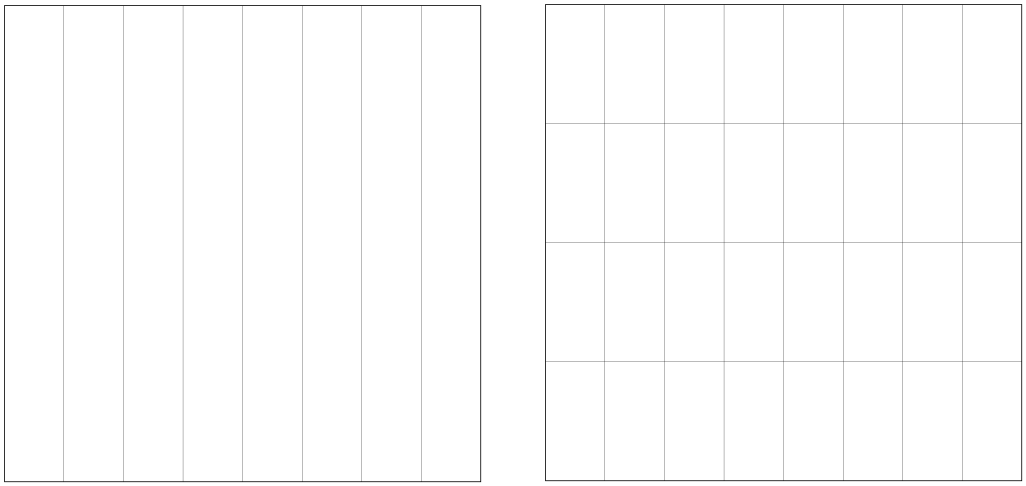


(b) k-d tree



(c) omnitree

Figure 4.1: Two hierarchical representations of the illustrative 2-dimensional mesh considered in Section 4.2.2. The adaptive mesh (a) is shown together with corresponding k-d tree (b) and omnitree (c). Leaf colors identify corresponding geometric cells.



(a) Omnitree mesh. (b) Cyclic k-d tree mesh.

Figure 4.2: Strongly anisotropic two-dimensional refinement considered in Section 4.2.3. The omnitree (a) and k-d tree (b) meshes are shown. Only leaf boundaries are drawn; colors and cell labels have been removed to make the additional k-d tree cuts visible.

nodes for the omnitree become thirty-one internal nodes and sixty-three total nodes for the k-d tree (see Table 4.1, line 2). The total topology is therefore 4.2 times larger. All omnitree leaves lie at depth three, while all k-d tree leaves lie at depth five.

This example shows that the difficulty is not an unfavorable balancing heuristic or a poor selection of split positions. All splits occur at the required dyadic midpoints. The inefficiency follows solely from coupling the axis to the scalar depth. Repeating one physically relevant direction forces subdivision of an orthogonal direction, and each forced subdivision duplicates every cell carried by the affected branch. The overhead is multiplicative, not additive, and it grows rapidly when more inactive directions must be crossed before the desired axis returns in the cycle.

In an adaptive spline computation, these extra leaves have several downstream effects. They enlarge the integration mesh, increase the number of element visits during error estimation, create fine-scale interfaces in directions where the solution is smooth, and may activate tensor-product functions associated with unnecessary directional levels. Even if a later algebraic stage can discard some contributions, the geometric representation has already paid for the surplus cells.

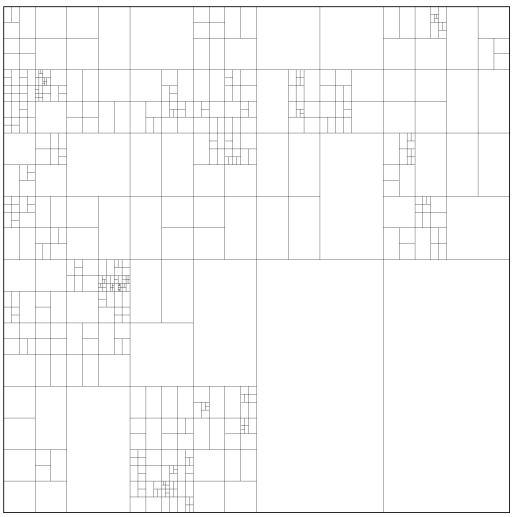


Figure 4.3: Fair random two-dimensional case considered in Section 4.2.4 with 501 leaves in both tree structures. The leaf partitions coincide

4.2.4 *Fair random meshes*

Random meshes were generated by repeatedly choosing an active omnitree leaf. In the fair mode, exactly one direction is refined and that direction is selected from the current floor $|\ell|$ so that it agrees with the cyclic k-d tree axis. Every refinement then replaces one leaf by two leaves in both structures. The final leaf counts are therefore identical by construction: 501 in two dimensions, 1501 in three dimensions, and 2001 in four dimensions. The comparison is reported on lines 3-5 of Table 4.1 (2D/3D/4D fair).

The two-dimensional partitions are shown in Figure 4.3. At page scale they are almost indistinguishable, as expected: fairness removes the geometric leaf inflation and makes the represented partition the same. The remaining differences in Table 4.1 are consequently attributable to topology and normalization.

In two dimensions, the binary k-d tree necessarily has 500 internal nodes and 1001 total nodes. Normalization reduces the omnitree to 312 internal and 813 total nodes. The reduction is 37.6% for internal nodes and 18.8% for the complete topology. The average leaf depth decreases from 11.02 to 6.19, or by 43.8%.

The effect remains stable as the dimension grows. With 1501 common leaves in three dimensions, the total-node count decreases from 3001 to 2375, and the average depth decreases from 13.22 to 5.78. With 2001 com-

mon leaves in four dimensions, the total-node count decreases from 4001 to 3181, and the average depth decreases from 13.88 to 5.87. The omnitree uses approximately 20% fewer total nodes in both higher-dimensional fair cases, while the mean traversal depth is reduced by 56.3% in three dimensions and 57.7% in four dimensions.

These results isolate a benefit that does not depend on avoiding extra mesh cells. Although each random refinement request is binary, the final normalized structure can recognize refinements common to sibling regions, promote them, and combine directions in higher-arity nodes. A binary k-d tree cannot contract those sequences because binary arity is part of its representation. Fairness therefore provides a controlled comparison: equal discretization size does not imply equal hierarchical overhead.

4.2.5 *Mesh produced by anisotropic function approximation*

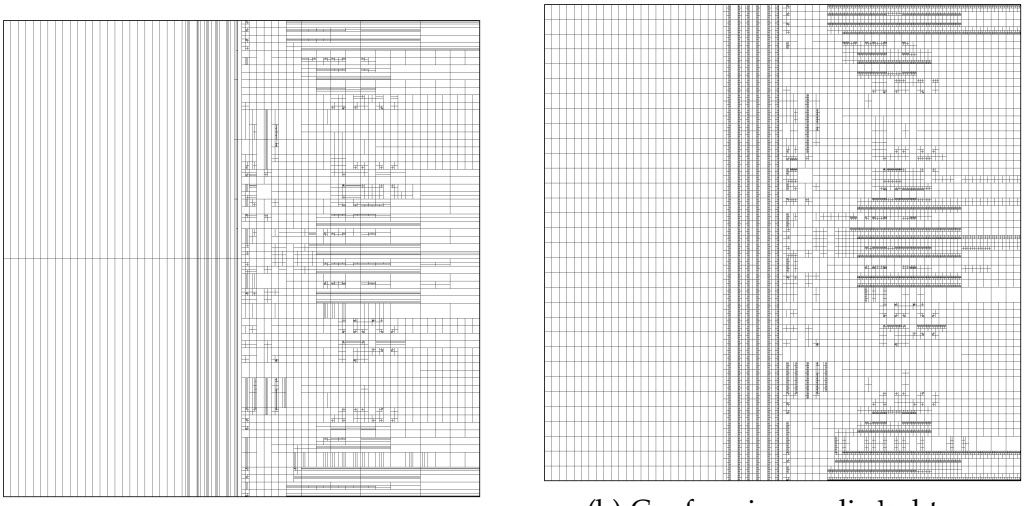
The second anisotropic example is not an artificial sequence of global strip refinements. Its mesh was produced by approximating the function 5.3

$$0.1\Big(B_0^7(x)\sin(120x)\sin(2\pi x) + 2B_1^7(x)\sin(120x)\sin(2\pi x) + B_7^7(x)\big[2-2(1+0.4\sin(60y))|\cos(2\pi y)|\big]\Big)$$

The exact approximation workflow is deferred to Chapter 5; the present comparison concerns only the tree structures generated by its directional refinement decisions. Figure 4.4 shows the two partitions.

The results for this example are reported on line 6 of Table 4.1 (2D unrestricted). The omnitree mesh contains 2695 leaves, whereas the converted k-d tree contains 15700. Thus the cyclic representation introduces 13005 additional active boxes and has 5.83 times as many leaves. Since the k-d tree is full and binary, it also requires 15699 internal nodes. The normalized omnitree requires only 1324 internal nodes. The total-node counts, 4019 and 31399, differ by a factor of 7.81; equivalently, the omnitree reduces total topology by 87.2% relative to the k-d tree.

Depth exhibits the same trend. Omnitree leaves range from level five to level nine and have average depth 7.24. k-d tree leaves range from depth ten to depth eighteen and have average depth 15.96. The k-d tree is therefore not merely wider because it contains more leaves; typical access paths are also substantially longer.



(a) Omnitree mesh.

(b) Conforming cyclic k-d tree refinement.

Figure 4.4: Adaptive meshes associated with the anisotropic function-approximation case considered in Section 4.2.5. Fine structures become visually dense at this scale, but the different amount of subdivisions remains apparent.

The left half of Figure 4.4 contains long, narrow regions, while the right half combines localized horizontal and vertical structures. An omnitree can preserve a long cell until refinement is needed in its short direction. The cyclic k-d tree must repeatedly cut the long direction whenever that axis reappears in the cycle. In regions where the preferred direction changes, the issue is not removed: the phase of the global axis cycle may still be misaligned with the next required local split.

These figures are close to the limit of a useful static two-dimensional drawing. Thousands of boundaries merge at page scale, and individual cells cannot all be resolved. The plots are retained because they still reveal the global anisotropic pattern; the numerical statistics provide the reliable quantitative comparison.

4.2.6 *Unrestricted random meshes in three and four dimensions*

In unrestricted mode, the selected leaf is assigned a uniformly selected nonzero direction mask. A refinement may therefore split any subset of the d directions. This process exercises the defining omnitree operation rather than restricting it to the k-d tree axis sequence. The converted k-d

tree must serialize simultaneous splits and insert missing axes whenever the local requested order disagrees with its depth cycle.

The results for this class of examples are reported in Table 4.1 (line 7-8). The three-dimensional omnitree contains 4241 leaves, 1546 internal nodes, and 5787 total nodes. The corresponding cyclic k-d tree grows to 876963 leaves and 1753925 total nodes. The leaf ratio is 0.48%, and the total-node ratio is 0.33%. Expressed as reductions relative to the k-d tree, the omnitree uses 99.52% fewer leaves and 99.67% fewer total nodes. Its average leaf depth is 10.22, compared with 36.82;

The unrestricted four-dimensional case contains more omnitree leaves, 8067, but fewer k-d tree leaves than the three-dimensional realization, 745659. The relevant comparison is within each pair. In four dimensions the k-d tree still has 92.43 times as many leaves and 147.44 times as many total nodes. Average depth rises from 8.95 in the omnitree to 38.19 in the k-d tree, and maximum depth rises from sixteen to forty-nine. The omnitree reductions are 98.92% in leaves and 99.32% in total nodes.

No mesh drawings are included for these cases. In three dimensions, a boundary plot of thousands of omnitree boxes is already difficult to interpret, and the corresponding k-d tree contains nearly nine hundred thousand leaves. In four dimensions there is no direct static spatial depiction without selecting projections or slices, which would hide part of the refinement state. Beyond the two-dimensional cases shown above, the structures are therefore too complex for a faithful and legible page-scale representation; Table 4.1 is the appropriate description.

The scale of the unrestricted results follows from two compounding mechanisms. First, one omnitree node can realize s simultaneous directional splits with 2^s children, whereas a binary tree needs s consecutive levels even when their order is favorable. Second, the cyclic order is frequently unfavorable, so irrelevant axes are inserted between required axes. Each inserted split acts on a region rather than on a symbolic path only: it creates real leaves that must be retained to keep a conforming binary partition. Repeating this process over randomly selected local regions produces the observed multiplicative growth.

4.2.7 *Overall assessment*

The examples distinguish three increasingly demanding situations.

1. In the preliminary example, both trees have the same leaves, but normalization reduces internal nodes and depth.

2. In the fair random examples, equality of leaf counts is enforced at larger scales. The omnitree still reduces total topology by about one fifth and mean depth by roughly one half in three and four dimensions.

3. In the strongly anisotropic and unrestricted examples, the k-d tree must also add geometric cells. Leaf and node ratios then range from moderate factors in two dimensions to two or three orders of magnitude in higher-dimensional random meshes.

In multidimensional data structures, k-d trees have long served as the de facto standard for organizing and querying points in Euclidean space. Their simplicity, strong theoretical grounding, and wide adoption in libraries and applications have made them a default choice for tasks such as nearest-neighbor search, range queries, and spatial indexing. The tree structure retains advantages of conceptual simplicity, fixed binary arity, and direct pointer-based traversal. For meshes whose requested directions already follow its cycle, it represents the leaves exactly and its node-count identity is predictable. These properties make it a useful reference implementation. They do not compensate, however, for the loss of directional freedom in the target applications. In fact,they inherit several structural limitations: they rely on axis-aligned splits, are sensitive to dimension ordering and data distribution, and can degrade significantly in performance as dimensionality increases or when data are highly anisotropic or clustered in complex ways.

LHB-spline refinement is formulated in a lattice precisely because different coordinates must be refined independently. A data structure that re-couples those coordinates through scalar depth works against that formulation.

The omnitree aligns more closely with the lattice hierarchy. Its node mask and an LHB level vector carry the same directional information, Morton-ordered leaves provide the active mesh consumed by the basis constructor, and normalization compresses common refinement without changing the partition. The comparisons show that this alignment is useful even when the leaves are held fixed and becomes decisive when anisotropy is strong or the dimension increases. For the implementation of Lattice Hierarchical B-splines, the omnitree is therefore used as the primary mesh representation.

4.3 REPRESENTATION OF LATTICE HIERARCHICAL MESHES AND LHB-SPLINES

The implementation described in this section converts an omnitree adaptive mesh into a LHB-spline basis. Three objects must be distinguished throughout the construction.

- The *adaptive mesh* is the complete partition represented by the DyAda discretization.
- A *leaf box* is one active cell of the adaptive mesh.
- A *basis grid* is the uniform tensor grid associated with one candidate B-spline level. It is a temporary grid used for testing functions and is not a replacement for the adaptive mesh.

The implementation first extracts the leaf boxes from the omnitree, constructs the candidate tensor-product bases, projects the leaf levels onto the grid of each candidate basis, and finally evaluates the LHB activity conditions on the complete support of every function.

4.3.1 *Construction pipeline*

The entry point is summarized in Listing 4.5.

```
discretization = dyada.Discretization(
    dyada.MortonOrderLinearization(),
    dyada.RefinementDescriptor.from_binary(
        num_dimensions,
        initial_mesh_binary,
    ),
)

max_refinement_level = max(
    discretization.descriptor.get_maximum_level()
)
leaf_boxes = list(
    discretization.get_all_boxes_level_indices()
)

knot_vectors = get_knot_vectors(
    max_refinement_level,
    polynomial_degree,
)
tensor_bases = get_tensor_bases(
```

```
    num_dimensions,
    knot_vectors,
)
active_functions = get_active_functions(
    tensor_bases,
    leaf_boxes,
)
```

Listing 4.5: Main stages of the LHB-spline construction.

The object `discretization` represents the whole adaptive mesh. In contrast, `leaf_boxes` contains only its active cells. For a leaf Q, `Q.d_level` is the multi-level that determines its directional sizes, while `Q.d_index` determines its position at that level.

The maximum component reached by the mesh is used to generate all univariate levels from zero to `max_refinement_level`. The resulting tensor bases are stored in a dictionary whose key is a complete level tuple. For example, the key $(3, 1)$ identifies a basis defined on eight uniform intervals in the first direction and two in the second.

4.3.2 *Knot vectors and tensor-product bases*

At univariate level r, `get_knot_vectors` creates 2^r uniform knot spans on $[0, 1]$. Since `linspace` already includes both endpoints, adding `polynomial_degree` further zeros and ones gives endpoint multiplicity `polynomial_degree + 1`. Consecutive levels are therefore related by dyadic knot insertion.

The function `get_tensor_bases` constructs all anisotropic tensor-product B-spline bases across the spatial dimensions. It computes the Cartesian product of the available refinement levels for each dimension and maps each level tuple (e.g., $(2, 0)$) to its corresponding tensor B-spline basis instantiated from the respective knot vectors.

To handle spline operations efficiently, an open-source C++ framework designed for Isogeometric Analysis called G+Smo (Geometry + Simulation Modules) [23] is used. The dimension-specific G+Smo constructor is selected dynamically, and each dictionary entry has the form

$$\texttt{level tuple} \longmapsto \texttt{tensor-product basis}.$$

This construction is exhaustive: if the largest mesh level is L, all tuples in $\{0, \ldots, L\}^d$ are considered. Some of the corresponding bases may contain no active function, but no potentially relevant lattice level is omitted.

4.3.3 *Projection of the adaptive mesh onto a basis grid*

The function `compute_ancestor_grid` is called once for every candidate basis level. Its parameter `target_levels` is the level tuple of the tensor basis currently being examined. The associated uniform grid has shape

$$(2^{k_1}, \ldots, 2^{k_d}),$$

where `target_levels` is $(k_1, \ldots, k_d)$.

Each leaf of the adaptive mesh must be expressed in the index system of this uniform grid. The relevant part of the mapping is shown in Listing 4.6.

```
for target_lvl, box_lvl, box_idx in zip(
    target_levels,
    box.d_level,
    box.d_index,
):
    if target_lvl >= box_lvl:
        diff = target_lvl - box_lvl
        start = box_idx << diff
        end = (box_idx + 1) << diff
    else:
        diff = box_lvl - target_lvl
        start = box_idx >> diff
        end = start + 1

    grid_ranges.append(range(start, end))
```

Listing 4.6: Mapping one adaptive leaf onto the grid of a candidate basis.

If the leaf is coarser than the basis grid in one direction, it covers several consecutive grid cells. A left shift expands its index to the finer resolution. If the leaf is finer, a right shift maps it to its unique ancestor cell on the coarser basis grid. The Cartesian product of the directional ranges gives all basis-grid cells covered by the leaf.

The resulting `ancestor_grid` does not store mesh cells. At every index, the update computes the componentwise minimum between the value already stored there and `box.d_level`. Thus, each entry records the componentwise minimum of the levels of all adaptive leaves mapped to that basis-grid cell.

The name `ancestor_grid` refers to this componentwise minimum. An entry such as $(2, 1)$ means that $(2, 1)$ is the closest common predecessor of the leaf levels represented at that location. It does not necessarily mean that a leaf of level $(2, 1)$ is present there.

4.3.4 *Support of a candidate B-spline*

For every tensor basis, `local_index` selects one B-spline. The call `tensor_basis.support(local_index)` returns its lower and upper support coordinate in every direction; transposition with the Python built-in function `zip` groups these values into `support_min_corner` and `support_max_corner`.

The support is the rectangular region where the selected function is non-zero. Its continuous bounds are converted into a tuple of NumPy slices on the basis grid. The lower bounds are rounded down and the upper bounds are rounded up, so the resulting `sub_tensor` contains the local level information beneath the complete support.

This grid is the connection between the two representations: adaptive leaves are projected onto the basis grid, and the B-spline support is expressed as a slice of the same grid. The activation test can consequently inspect the mesh levels beneath the support without performing a geometric intersection against every leaf for every function.

4.3.5 *Activity conditions*

Let `basis_levels` denote the level of the candidate function. Listing 4.7 contains the two activity conditions. The loop first rejects the function if its level is finer than the local mesh in any direction at any point of its support. At the same time, it computes the closest common predecessor of all levels observed under the support. The final comparison applies the second condition.

```
closest_common_ancestor = list(sub_tensor.flat[0])

for cell_box_level in sub_tensor.flat:
    if any(
        func_l > box_l
        for func_l, box_l in zip(
            basis_levels,
            cell_box_level,
        )
    ):
        return False

    closest_common_ancestor = [
        min(cur_min, box_l)
        for cur_min, box_l in zip(
            closest_common_ancestor,
```

```
            cell_box_level,
        )
    ]

return all(
    anc_l <= func_l
    for anc_l, func_l in zip(
        closest_common_ancestor,
        basis_levels,
    )
)
```

Listing 4.7: Combined implementation of the two LHB activity conditions.

If the loop succeeds, `basis_levels` is componentwise smaller than or equal to every local level and therefore also smaller than or equal to `closest_common_ancestor`. The final condition imposes the converse relation. Together, the two conditions imply equality between the function level and the closest common predecessor over its support. Geometrically, the support is valid at the candidate level but is not completely contained in any single immediate child subdomain.

The first condition already implies

$$\texttt{basis_levels} \preceq \texttt{closest_common_ancestor},$$

therefore, requiring the function level to be strictly greater than that ancestor would contradict the first condition and would reject every candidate. Reversing the strict inequality would instead accept functions whose supports are completely covered by finer levels, which is also incorrect.

4.3.6 *Active-function collection and output*

The function `get_active_functions` iterates over the dictionary of tensor bases. For each level it constructs the corresponding grid information once, then examines all local basis functions. A consecutive global identifier is assigned only when the activity and non-ghost tests succeed.

The returned dictionary has the form

$$\texttt{global identifier} \longmapsto (\texttt{basis level}, \texttt{local basis index}).$$

For example, `active_functions[17] = ((2, 1), 8)` means that global function 17 is local function 8 of the tensor-product basis at level $(2, 1)$. The output contains neither coefficients nor duplicated B-spline objects.

The level tuple identifies the correct entry of `tensor_bases`, and the local index identifies the function inside that basis.

The complete computational flow is therefore

$$\begin{aligned} &\texttt{omnitree descriptor} \longrightarrow \texttt{adaptive adaptive mesh} \longrightarrow \\ \longrightarrow\ &\texttt{candidate tensor bases} \longrightarrow \texttt{level data on each basis grid} \longrightarrow \\ &\longrightarrow \texttt{support tests} \longrightarrow \texttt{active LHB functions}. \end{aligned}$$

5

ADAPTIVE APPROXIMATION WITH LHB-SPLINES

The previous chapter described the construction of the active LHB-spline basis on a given lattice hierarchical mesh. This chapter studies how that basis behaves inside an adaptive least-squares approximation procedure. The main question is whether directional refinement can reproduce a prescribed target with fewer degrees of freedom than classical hierarchical refinement. To answer it, the HB-spline and LHB-spline experiments use the same target data, sampling grid, spline degree, initial discretization, least-squares solver and marking tolerance. Their essential difference lies in the refinement mask: HB-spline refinement splits every marked element in all parametric directions, whereas LHB-spline refinement may split it only in the directions selected from the local variation of the target. This adaptive organization follows the general LHB-spline refinement framework introduced in [6], while the precise numerical criteria described below are those implemented in the supplied Python code.

The distinction is particularly relevant for thin, oriented features. A ridge that changes rapidly across its width but remains almost constant along its length does not require the same resolution in both directions. An isotropic mesh nevertheless reduces both element sizes simultaneously. A lattice hierarchical mesh can instead create elongated elements, concentrating new basis functions in the direction that contributes most to the unresolved variation. The experiments in Section 5.3 show both the potential of this mechanism and the limitations observed in the available computations.

5.1 ADAPTIVE SPLINE REFINEMENT

Let $\Omega = [0,1]^d$ and let $\mathcal{Q}_k$ be the set of active elements after the k-th adaptive step. An element $Q \in \mathcal{Q}_k$ is identified by the multi-level

$$\ell(Q) = \big(\ell_1(Q), \dots, \ell_d(Q)\big).$$

Its size in direction j equals to $2^{-\ell_j(Q)}$. For a classic hierarchical mesh all components of the level are equal. Consequently, whenever an element is refined, its size is reduced in every direction. In a Lattice hierarchical mesh the level components may differ, so the same element can be fine in one direction and coarse in another. This is the geometric mechanism that produces the thin rectangles visible in the two surface-fitting experiments. The bases are rebuilt after every refinement, since changing the leaf mesh can activate new levels, remove coarse functions, and alter which tensor-product B-splines satisfy the active basis criterion described in Chapter 4.

The adaptive computation follows the same sequence at every iteration. First, the active boxes and the largest level currently present are read from the discretization. The required knot vectors and tensor-product spline bases are then generated, and the active hierarchical functions are selected. Second, these functions are evaluated at the sampling points and assembled into the collocation matrix. A least-squares solver supplies the coefficients in 5.1. Third, the residual is evaluated on the same sampling grid. Its values are used both to report the quality of the fit and to identify the elements that still contain insufficiently resolved points. Finally, the marked elements are refined and the entire procedure is repeated on the new mesh.

For HB-splines, the last step is immediate. In two parametric dimensions every marked element receives the mask "11"; in three parametric dimensions it receives "111". For LHB-splines, a direction-selection step is inserted before refinement. A two-dimensional marked element can receive "10", "01", or "11"; in three dimensions the same idea produces a three-bit mask. The mask determines which components of $\ell(Q)$ are increased.

This comparison is controlled at the approximation level. In both the experiments, HB-splines and LHB-splines call the same routines for grid generation, basis evaluation, sparse matrix assembly, and the same iterative `lsmr` solver. A comparable marking algorithm has not been implemented yet, as it was beyond the main scope of this thesis, and is therefore left for future work.

The process can be summarized as

$$\begin{aligned}\text{mesh} &\longrightarrow \text{active basis} \longrightarrow \text{least-squares fit} \longrightarrow \\ &\longrightarrow \text{residual} \longrightarrow \text{marking} \longrightarrow \text{refinement}\end{aligned}$$

For LHB-splines, the refinement direction is selected between the marking and refinement stages. The next section explains these decisions in the same order in which they appear in the code.

5.2 ERROR ESTIMATION AND ANISOTROPIC REFINEMENT

The adaptive strategy consists of three separate components: the least-squares problem, which determines the spline coefficients, the reported MSE, which summarizes the overall quality of the resulting fit, the pointwise residual and the gradient decide where and in which direction the mesh is refined.

5.2.1 *Least-squares fit and reported error*

Let $\mathbf{x}_1, \ldots, \mathbf{x}_N$ be the uniform sampling points and let $y_q = u(\mathbf{x}_q)$ be the corresponding target values, with q the sampling point index. Once the active basis has been constructed, the collocation matrix is defined by

$$(A_k)_{q\alpha} = B_{k\alpha}(\mathbf{x}_q),$$

where k is the iteration index, and α is the basis function index. The coefficient vector is computed by solving

$$\mathbf{c}_k \in \arg\min_{\mathbf{c}} |A_k \mathbf{c} - \mathbf{y}|_2^2. \tag{5.1}$$

Thus, the least-squares formulation in 5.1 minimizes the sum of squared discrepancies at the sampled points.

After the coefficients have been found, the absolute residual at a sample is

$$e_{k,q} = |y_q - u_k(\mathbf{x}_q)|,$$

which measures the difference between the exact target value y_q and the approximation $u_k(\mathbf{x}_q)$ at the q-th point. The result consists of two global summaries:

$$\mathrm{MSE}_k = \frac{1}{N} \sum_{q=1}^{N} e_{k,q}^2, \qquad E_{\max,k} = \max_{1 \leqslant q \leqslant N} e_{k,q}.$$

The MSE is used in the convergence plots because it gives one scalar value for each approximation and can be compared directly with the number of active basis functions. The maximum error has a different role: it reveals whether a small global average hides a localized point at which the approximation is still poor.

5.2.2 *How an element is marked*

The marking rule implemented is pointwise. The code scans all sampled points for which $e_{k,q} > \varepsilon_{\text{ref}}$.

An active element is added to the marked set if at least one of its sampled residuals exceeds the threshold ε_{ref}. If no sampled point violates the threshold, no refinement is requested. The criterion is local: it asks whether each region is adequately represented, not only whether the average error over the complete domain is small. Points lying exactly on a boundary shared by several active boxes are not taken into consideration.

5.2.3 *How LHB-spline refinement directions are selected*

The target values are differentiated on the uniform grid before the adaptive loop starts. In two dimensions this produces approximations of $\partial_x u$ and $\partial_y u$; in three dimensions it also produces $\partial_z u$. The absolute value of each component is stored. For a marked element Q, the code then averages every absolute gradient component over the sampling points assigned to that element. Denoting these averages by

$$g_{Q,j} = \frac{1}{|I_Q|} \sum_{q \in I_Q} |D_j^h u(\mathbf{x}_q)|,$$

where $D_j^h u(\mathbf{x}_q)$ denotes the discrete derivative along direction j, I_Q is the index set of sampling points inside Q, and $|I_Q|$ is its cardinality. The largest component is

$$g_{Q,\max} = \max_j g_{Q,j}.$$

Direction j is then selected for refinement if

$$g_{Q,j} \geqslant \varepsilon_{\text{dir}} g_{Q,\max}, \tag{5.2}$$

where $\varepsilon_{\text{dir}} \in (0, 1]$ is a prescribed directional tolerance parameter controlling how anisotropic the refinement can be. The experiments use

$\varepsilon_{\text{dir}} = 0.3$. According to (5.2), a component is therefore selected if it is at least 30% of the largest component in the same element. If only one component is significant, the element is split in one direction. If two or more components are comparable, all of them are selected. As a result, when the entire average gradient vanishes, every direction is selected for refinement as a safe fallback.

The gradient does not decide whether an element needs refinement. That decision has already been made from the residual. The gradient only decides the orientation of the split. This separation makes the adaptive logic intuitive: the residual identifies a region that is not sufficiently resolved, while the gradient indicates how the target varies inside that region.

5.2.4 *Stopping the adaptive loop*

The marking tolerance is a global stopping criteria in the supplied scripts and the loop stops under the following circumstances:

- The prescribed maximum number of iterations is reached. Each current approximation is saved before this test, so the last computed state remains available for the convergence history.
- No element is marked. This means that no point has an absolute residual larger than `tol`.

The discussion below refers to the "last available iteration" in any case.

5.3 NUMERICAL EXAMPLES

Two experiments are considered. They approximate scalar functions on $[0, 1]^2$; their graphs are surfaces in three-dimensional space. The two surface-fitting experiments use cubic splines, a 500×500 sampling grid, the pointwise marking tolerance 10^{-3}, and at most ten iterations. The LHB-spline directional threshold is 0.3. Table 5.1 summarizes the settings read directly from the parameter files.

Table 5.1: Parameters used in the numerical experiments.

Parameter	Value
Parametric dimension	2
Spline degree	3
Samples per direction	500
Marking tolerance	10^{-3}
Maximum iterations	10
LHB-spline direction threshold	0.3

5.3.1 *First bivariate case: mixed directional features*

The first target is the oscillatory surface used to test how the refinement direction changes across the domain. Let $B_i^7(x) = \binom{7}{i} x^i (1-x)^{7-i}$ denote the degree-seven Bernstein polynomials. The target is

$$u_1(x,y) = 0.1\Big(B_0^7(x)\sin(120x)\sin(2\pi x) + 2B_1^7(x)\sin(120x)\sin(2\pi x) + B_7^7(x)\big[2 - 2(1 + 0.4\sin(60y))|\cos(2\pi y)|\big]\Big). \tag{5.3}$$

The function in (5.3) is plotted on Figure 5.1, and it does not have one uniform orientation throughout the square. Its left portion is dominated by rapid variation in x, while the contribution depending on y becomes important towards the right. It therefore tests whether LHB refinement remains anisotropic where one component dominates and becomes more isotropic where both components are relevant.

Figures 5.2, 5.3, and 5.4 show three representative stages of each adaptive history. The stages are intended to show the evolution within each method.

The early stages of the two histories are close. For example, HB-splines at iteration 2 have 361 DoFs with MSE 8.7491×10^{-4}, while LHB-spline at iteration 2 has 223 DoFs with MSE 8.7511×10^{-4}. A more visible difference appears later: HB-splines at iteration 6 reaches an MSE 4.5189×10^{-8} with 8102 DoFs, whereas LHB-spline at iteration 5 reaches a comparable value of 9.7682×10^{-8} with only 1516 DoFs. The LHB-spline error is slightly larger, but the basis contains about 81.3% fewer functions.

After this point, the available LHB-spline history experiences a sharp error reduction, reaching MSE$= 9.7682 \times 10^{-8}$ with 1516 DoFs.

Figure 5.5 contains the complete convergence data.

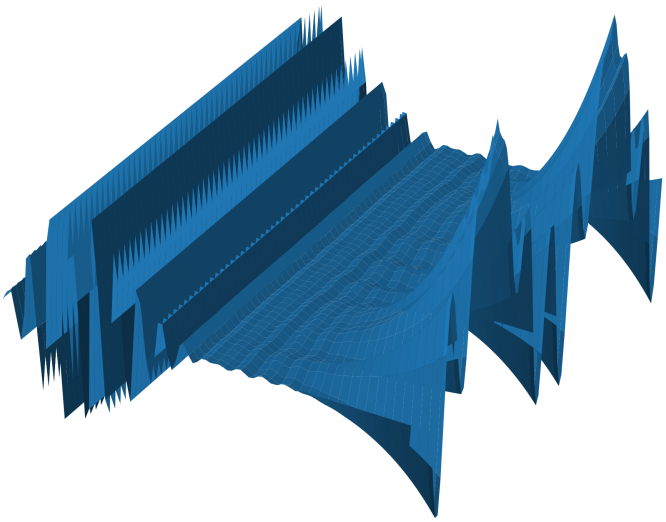


Figure 5.1: First numerical example function 5.3

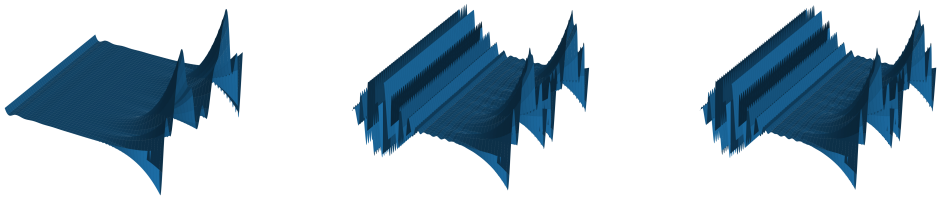
	DoFs		MSE		Maximum error	
Iter.	HB-spline	LHB-spline	HB-spline	LHB-spline	HB-spline	LHB-spline
1	121	121	9.2058×10^{-4}	9.2058×10^{-4}	9.1500×10^{-2}	9.1498×10^{-2}
2	361	223	8.7491×10^{-4}	8.7511×10^{-4}	9.2088×10^{-2}	9.2086×10^{-2}
3	1225	477	7.6262×10^{-4}	7.6298×10^{-4}	1.0414×10^{-1}	1.0413×10^{-1}
4	3967	1182	1.1293×10^{-6}	1.2229×10^{-6}	3.4751×10^{-3}	3.4756×10^{-3}
5	8099	1516	4.5426×10^{-8}	9.7682×10^{-8}	1.7749×10^{-3}	2.5177×10^{-3}
6	8102	1538	4.5189×10^{-8}	9.3469×10^{-8}	1.7222×10^{-3}	2.5326×10^{-3}
7	8130	1592	4.4815×10^{-8}	9.0946×10^{-8}	1.6729×10^{-3}	2.4430×10^{-3}

Table 5.2: Comparison between HB-spline and LHB-spline across iterations for the mixed-feature surface.

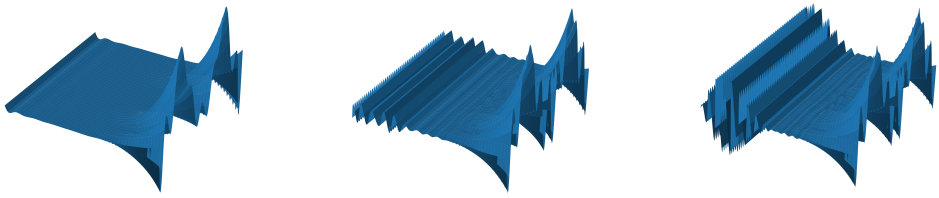
5.3.2 *Second bivariate case: an axis-aligned ridge*

The second target is a ridge centered at $x_c = 0.5$ with width $w = 0.1$ and steepness $\sigma = 100$:

$$u_2(x, y) = \frac{1}{4}\left[1 + \tanh\left(\sigma\left(x - x_c + \frac{w}{2}\right)\right)\right]\left[1 + \tanh\left(\sigma\left(x_c - x + \frac{w}{2}\right)\right)\right].$$



(a) HB, 361 DoFs (b) HB, 3967 DoFs (c) HB, 8102 DoFs



(d) LHB, 223 DoFs (e) LHB, 477 DoFs (f) LHB, 1516 DoFs

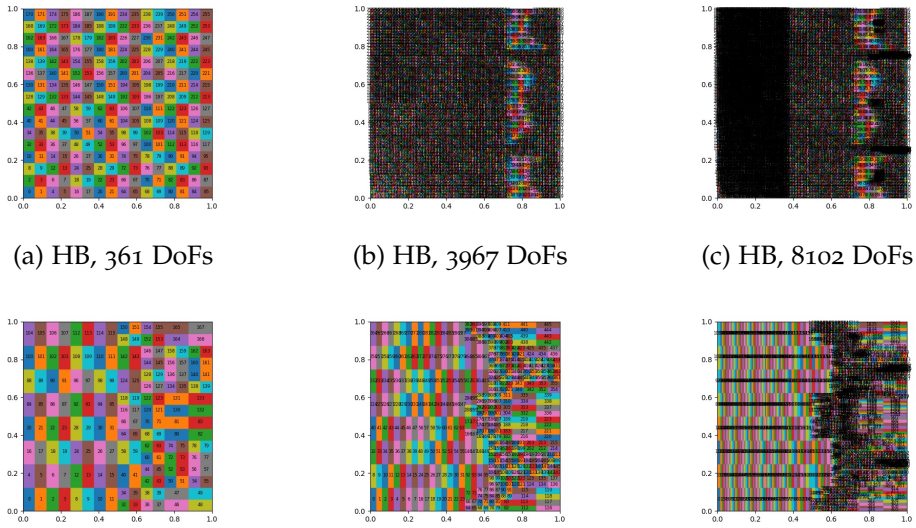
Figure 5.2: Evolution of the least-squares approximation for the mixed-feature surface. The first row shows HB-spline refinement and the second row LHB-spline refinement.

(5.4)

The function in (5.4) is independent of y. Consequently, all relevant variation is in the x direction. This is the clearest setting in which to compare isotropic and directional refinement: HB-spline refinement adds resolution in both x and y, whereas the LHB-spline gradient rule selects only x throughout the ridge transitions.

Figures 5.7, 5.8, and 5.9 reveal the source of the difference in basis size: the HB-spline mesh becomes fine in both coordinates in the central strip, while the LHB-spline mesh is refined primarily across the ridge and preserves long cells in the direction along which the target is constant.

This example gives the cleanest quantitative comparison because several HB and LHB-spline iterations have essentially the same recorded MSE and maximum error. At MSE 3.0501×10^{-3}, HB-spline uses 361 DoFs and LHB-spline uses 273 DoFs, a reduction of about 24.4%. At MSE 5.8566×10^{-5}, the counts are 2209 and 537, respectively, corresponding to about 75.7% fewer DoFs. At MSE 8.6575×10^{-10} (versus 8.8951×10^{-10}



(a) HB, 361 DoFs (b) HB, 3967 DoFs (c) HB, 8102 DoFs

(d) LHB, 223 DoFs (e) LHB, 477 DoFs (f) LHB, 1516 DoFs

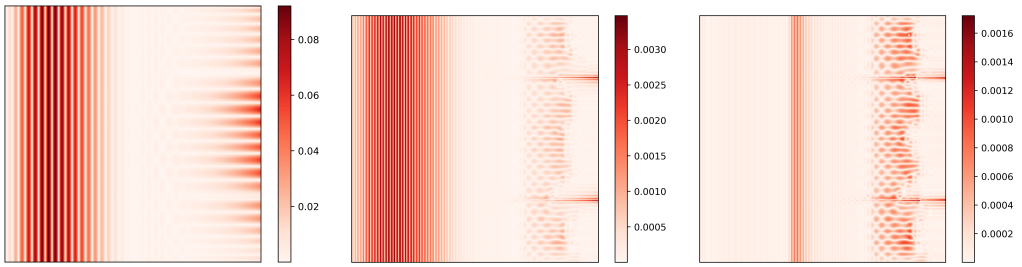
Figure 5.3: Evolution of the active meshes for the mixed-feature surface. HB refinement produces square cells, whereas the LHB-spline mesh contains elongated cells where the directional indicator selects a single component.

for HB), HB-spline uses 8245 DoFs and LHB-spline only 845 DoFs. The reduction is approximately 89.8%, while the maximum absolute errors agree to the displayed precision.

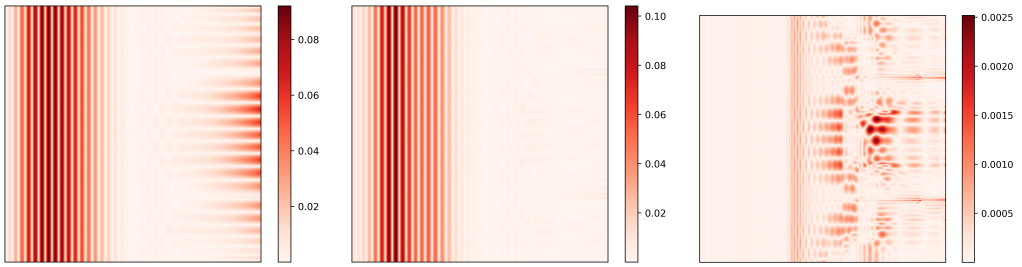
The equality of the errors is consistent with the structure of the target. Both spaces acquire essentially the same resolution across the ridge; HB introduces additional resolution along the y direction, which does not improve a function that is constant in y. LHB-spline retains the approximation power relevant to the target without paying the same tensor-product growth in the inactive direction.

The final LHB-spline points go beyond the available HB-spline history. Iteration 6 reaches MSE 8.6575×10^{-10} with 845 DoFs and maximum error 1.9857×10^{-4}. Since no HB-spline point at a comparable error was supplied, this final result should be read as the endpoint of the LHB-spline experiment rather than as a matched-error HB–LHB-spline comparison.

The full history is displayed in Figure 5.10.



(a) HB, 361 DoFs (b) HB, 3967 DoFs (c) HB, 8102 DoFs



(d) LHB, 223 DoFs (e) LHB, 477 DoFs (f) LHB, 1516 DoFs

Figure 5.4: Evolution of the sampled absolute error for the mixed-feature surface. Each panel uses the color scale generated for that iteration.

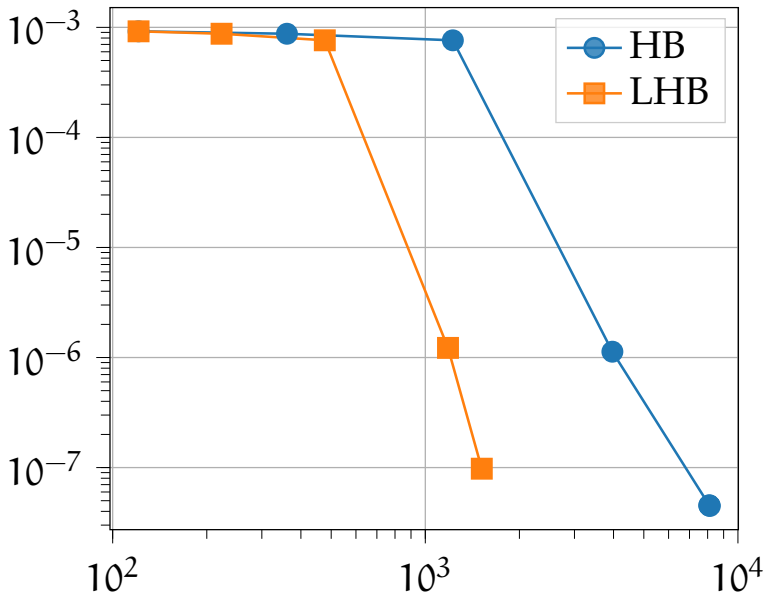


Figure 5.5: MSE against degrees of freedom for the first surface-fitting experiment.

5.3.3 *Overall interpretation*

Taken together, the experiments show that the benefit of lattice hierarchical refinement depends strongly on the directional structure of the

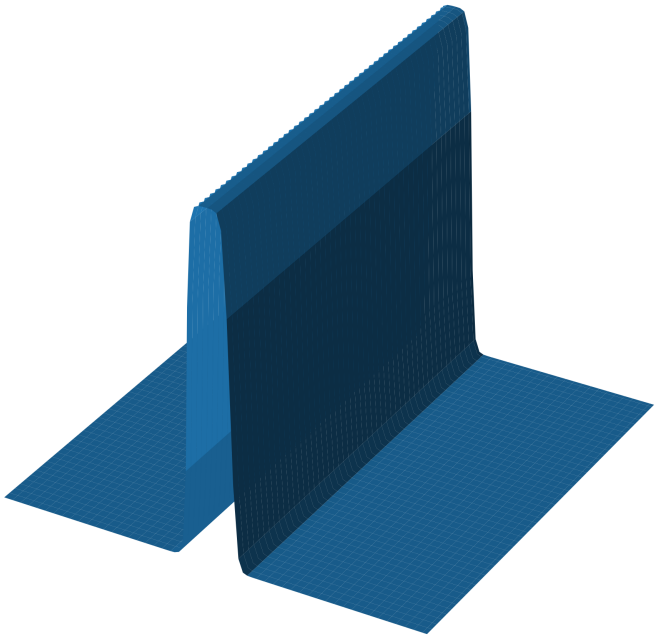
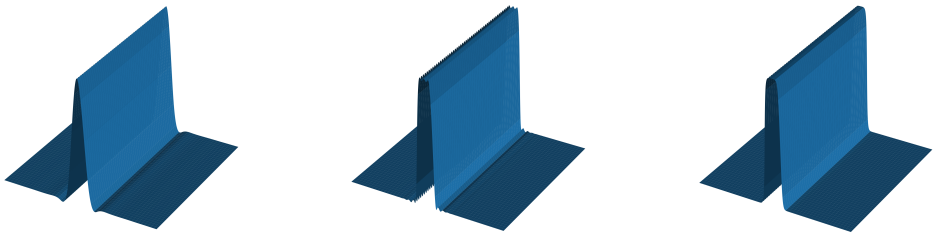


Figure 5.6: Second numerical example function 5.4

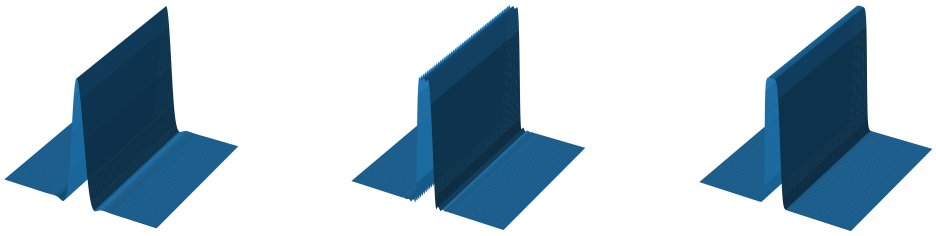
Iter.	DoFs		MSE		Maximum error	
	HB-spline	LHB-spline	HB-spline	LHB-spline	HB-spline	LHB-spline
1	121	121	1.5288×10^{-2}	1.5288×10^{-2}	3.9915×10^{-1}	3.9915×10^{-1}
2	361	273	3.0501×10^{-3}	3.0501×10^{-3}	2.4232×10^{-1}	2.4232×10^{-1}
3	721	361	3.5202×10^{-4}	3.5202×10^{-4}	8.9400×10^{-2}	8.9395×10^{-2}
4	2209	537	5.8566×10^{-5}	5.8566×10^{-5}	3.8860×10^{-2}	3.8848×10^{-2}
5	5917	757	8.1582×10^{-8}	8.1582×10^{-8}	2.0632×10^{-3}	2.0619×10^{-3}
6	8245	845	8.8951×10^{-10}	8.6575×10^{-10}	3.4681×10^{-4}	1.9857×10^{-4}

Table 5.3: Comparison between HB-spline and LHB-spline across iterations for the axis-aligned ridge

target. The axis-aligned ridge is the most favorable case. There, HB-spline and LHB produce the same recorded errors at several stages, while LHB-spline avoids refining the direction in which the function is constant. The



(a) HB, 361 DoFs (b) HB, 2209 DoFs (c) HB, 8245 DoFs

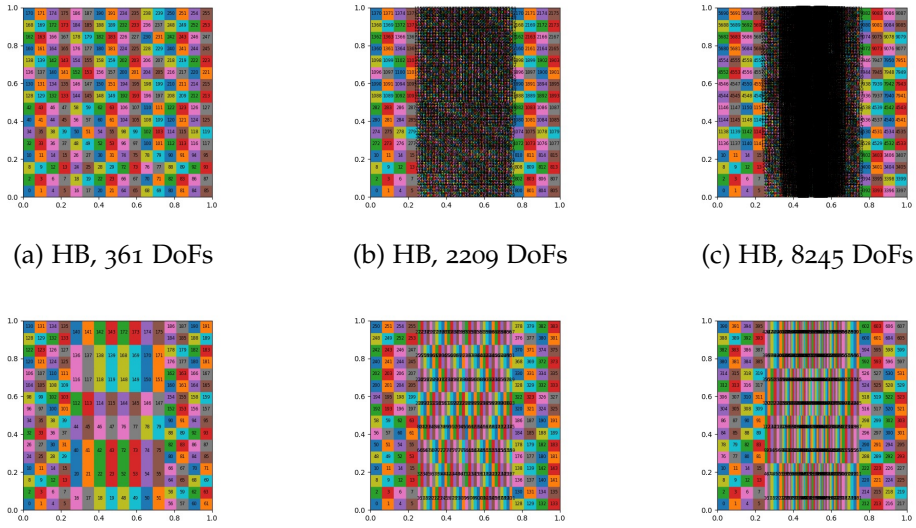


(d) LHB, 273 DoFs (e) LHB, 537 DoFs (f) LHB, 845 DoFs

Figure 5.7: Evolution of the least-squares approximation for the axis-aligned ridge.

reduction grows from about 24% to nearly 90% over the matched part of the history.

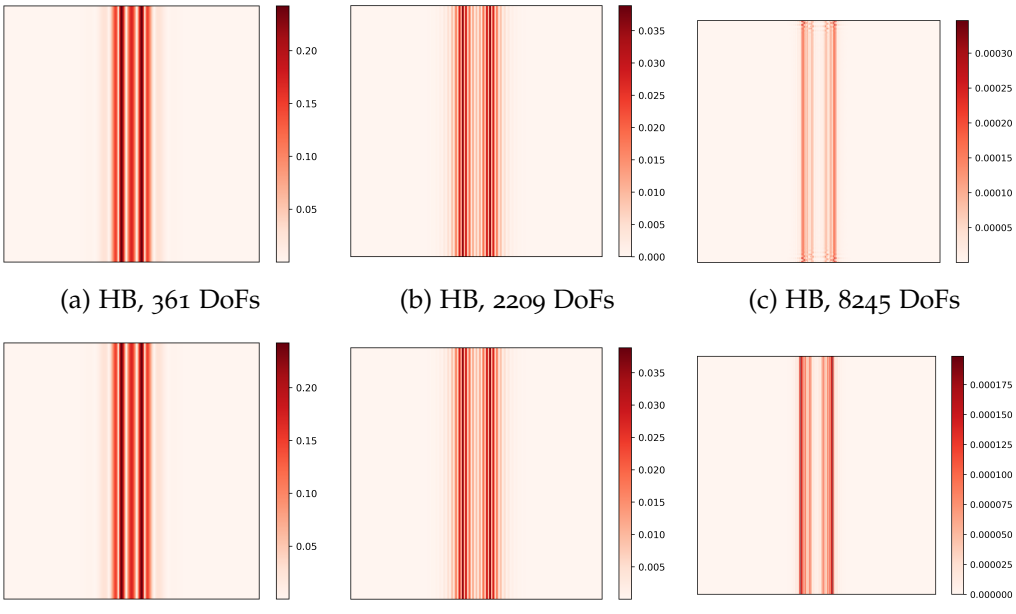
The mixed-feature surface is more complicated. At coarse accuracy the two methods have similar basis sizes, but LHB-spline begins to benefit when the mesh can separate regions with different preferred orientations. The sharp decrease in the later LHB-spline error is promising; nevertheless, the absence of HB-spline points at the same error prevents a complete matched-error comparison at the finest levels.



(a) HB, 361 DoFs (b) HB, 2209 DoFs (c) HB, 8245 DoFs

(d) LHB, 273 DoFs (e) LHB, 537 DoFs (f) LHB, 845 DoFs

Figure 5.8: Evolution of the active meshes for the axis-aligned ridge. The LHB-spline mesh concentrates thin vertical cells around the transition region.



(a) HB, 361 DoFs (b) HB, 2209 DoFs (c) HB, 8245 DoFs

(d) LHB, 273 DoFs (e) LHB, 537 DoFs (f) LHB, 845 DoFs

Figure 5.9: Evolution of the sampled absolute error for the axis-aligned ridge.

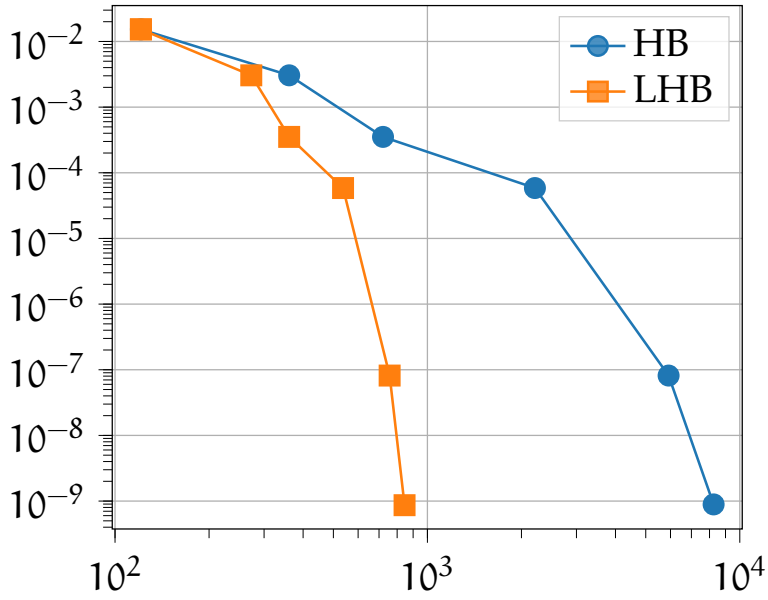


Figure 5.10: MSE against degrees of freedom for the axis-aligned ridge.

6

CONCLUSIONS

This thesis investigated the algorithmic, structural, and numerical foundations of directional adaptive spline approximation, overcoming the over-refinement limitations inherent to classical isotropic hierarchical refinement (HB-splines). Conventional isotropic approaches force uniform dyadic splits across all coordinate axes simultaneously, generating redundant degrees of freedom (DoFs) in the presence of strongly oriented features or interfaces. To solve this inefficiency, a complete computational pipeline based on Lattice Hierarchical B-splines (LHB-splines) was implemented and evaluated, decoupling directional resolutions to enable independent coordinate refinements.

6.1 KEY FINDINGS

The comparative analyses and numerical evaluations led to two principal conclusions:

- The structural comparison between normalized omnitrees (via the `DyAda` framework) and k-d trees demonstrated the severe overhead imposed by rigid splitting rules. k-d trees, constrained by scalar depth, suffered from severe topological expansion in multi-dimensional anisotropic settings. In contrast, normalized omnitrees significantly reduced average leaf depth and overall topology, proving to be the optimal underlying data structure for LHB-splines.
- Combining residual-based error indicators with local gradient estimates for direction selection proved highly effective. LHB refinement achieved a drastic reduction in degrees of freedom compared to isotropic HB-splines for equivalent target accuracy, tailoring the element geometry to directional features.

6.2 IMPACT AND FUTURE DIRECTIONS

Integrating normalized omnitrees with partially nested LHB-spline spaces provides a scalable computational architecture for anisotropic refinement. The open-source Python implementation (`lhb-python` [8]) bridges the gap between abstract lattice-spline theory and practical computational implementation. The ability to isolate localized directional variations while maintaining minimal tree depth makes this framework directly applicable to computationally demanding domains, including Isogeometric Analysis (IGA) [10, 11], which consists of efficient discretization of partial differential equations (PDEs) [2, 7] involving sharp boundary layers, shock waves, or highly localized stress concentrations [14], High-Dimensional Data Fitting [1, 17, 38], and Computational Fluid Dynamics (CFD) [3, 21, 34].

In conclusion, this thesis confirms the practical viability, mathematical integrity, and robust numerical performance of anisotropic hierarchical spline refinement, establishing LHB-splines and normalized omnitrees as a highly efficient framework for modern multidimensional computational science.

BIBLIOGRAPHY

[1] Andrew R. Barron. Universal approximation bounds for superpositions of a sigmoidal function. *IEEE Transactions on Information Theory*, 39(3):930–945, 1993.

[2] Yuri Bazilevs, Victor M. Calo, J. Austin Cottrell, Thomas J. R. Hughes, Alessandro Reali, and Guglielmo Scovazzi. Isogeometric analysis of turbulence and fluid–structure interaction. *Computational Mechanics*, 43(1):143–150, 2008.

[3] Yuri Bazilevs, Kenji Takizawa, and Tayfun E. Tezduyar. *Computational Fluid Dynamics: Effective Computations through Isogeometric Analysis and Meshfree Methods*. John Wiley & Sons, 2013.

[4] Jon Louis Bentley. Multidimensional binary search trees used for associative searching. *Comput. Sci.*, 18(9):509–517, 1975.

[5] Paul J Besl and Neil D McKay. A method for registration of 3-D shapes. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 14(2):239–256, 1992.

[6] Cesare Bracco, Carlotta Giannelli, Tadej Kandûc, and Hugo M. Verhelst. Lattice hierarchical B-splines. preprint, 2026.

[7] Alexander N. Brooks and Thomas J. R. Hughes. Streamline upwind/Petrov-Galerkin formulations for convection dominated flows with particular emphasis on the incompressible Navier-Stokes equations. *Computer Methods in Applied Mechanics and Engineering*, 32(1-3):199–259, 1982.

[8] CosimoMatassini. lhb-python. `https://github.com/CosimoMatassini/lhb-python`, 2026.

[9] CosimoMatassini. plot-bspline-basis. `https://github.com/CosimoMatassini/plot-bspline-basis`, 2026.

[10] J. Cottrell, Thomas Hughes, and Yuri Bazilevs. *Isogeometric Analysis: Toward integration of CAD and FEA*. John Wiley & Sons, 09 2009.

[11] J. Austin Cottrell, Thomas J. R. Hughes, and Yuri Bazilevs. *Isogeometric Analysis: Toward Integration of CAD and FEA*. John Wiley & Sons, 2009.

[12] Carl De Boor. *A Practical Guide to Splines*, volume 27. Springer-Verlag, New York, 1978.

[13] Tor Dokken, Tom Lyche, and Kjell Fredrik Pettersen. Polynomial splines over locally refined box-partitions. *Computer Aided Geometric Design*, 30(3):331–356, 2013.

[14] Alexander Düster, H. Bröker, and Ernst Rank. High-order finite elements for the analysis of localized stress concentrations and boundary layers. *Computational Mechanics*, 43(2):253–265, 2008.

[15] Norbert Engleitner and Bert Jüttler. Patchwork B-spline refinement. *Computer-Aided Design*, 90:168–179, 2017.

[16] R. A. Finkel and J. L. Bentley. Quad trees a data structure for retrieval on composite keys. *Acta Informatica*, 4:1–9, 1974.

[17] Jerome H. Friedman. Multivariate adaptive regression splines. *The Annals of Statistics*, 19(1):1–67, 1991.

[18] Jerome H Friedman, Jon Louis Bentley, and Raphael Ari Finkel. An algorithm for finding best matches in logarithmic expected time. *ACM Transactions on Mathematical Software (TOMS)*, 3(3):209–226, 1977.

[19] Carlotta Giannelli, Bert Jüttler, and Hendrik Speleers. THB-splines: The truncated basis for hierarchical splines. *Computer Aided Geometric Design*, 29(7):485–498, 2012.

[20] Carlotta Giannelli and Hendrik Speleers. *Advanced methods for geometric modeling and numerical simulation*. Springer, 2019.

[21] Francis H. Harlow and J. Eddie Welch. Numerical calculation of time-dependent viscous incompressible flow of fluid with free surface. *The Physics of Fluids*, 8(12):2182–2189, 1965.

[22] T.J.R. Hughes, J.A. Cottrell, and Y. Bazilevs. Isogeometric analysis: CAD, finite elements, NURBS, exact geometry and mesh refinement. *Computer Methods in Applied Mechanics and Engineering*, 194(39):4135–4195, 2005.

[23] Bert Jüttler, Ulrich Langer, Angelos Mantzaflaris, Stephen E Moore, and Walter Zulehner. Geometry+ simulation modules: Implementing isogeometric analysis. *PAMM*, 14:961–962, 2014.

[24] Donald Ervin Knuth. *The Art of Computer Programming, Volume 1: Fundamental Algorithms*. Addison-Wesley, 1968.

[25] R. Kraft. Adaptive and linearly independent multilevel B-splines, Surface fitting and multiresolution methods, 1997.

[26] Donald Meagher. Geometric modeling using octree encoding. *Computer Graphics and Image Processing*, 19(2):129–147, 1982.

[27] Donald J. R. Meagher. *Octree encoding: A new technique for the representation, manipulation and display of arbitrary 3-d objects by computer*. Electrical and Systems Engineering Department Rensselaer Polytechnic Institute, 1980.

[28] Guy M Morton. *A computer oriented geodetic data base and a new technique in file sequencing*. International Business Machines Company, 1966.

[29] O.J. Murphy and S.M. Selkow. The efficiency of using k-d trees for finding nearest neighbors in discrete space. *Information Processing Letters*, 23(4):215–218, 1986.

[30] Zienkiewicz O.C, R. Taylor, and J Zhu. *The Finite Element Method: Its Basis and Fundamentals*, volume I. Elsevier, 01 2005.

[31] Les Piegl and Wayne Tiller. *The NURBS Book*. Found. Comput. Math.. Springer-Verlag, Berlin, Heidelberg, 2nd edition, 1997.

[32] Theresa Pollinger. DyAda. `https://github.com/freifrauvonbleifrei/DyAda`.

[33] Theresa Pollinger, Masado Ishii, and Jens Domke. The beauty of anisotropic mesh refinement: Omnitrees for efficient dyadic discretizations. *Math. Comput. Simulat.*, 2026.

[34] Patrick J. Roache. *Computational Fluid Dynamics*. Hermosa Publishers, 1972.

[35] Hanan Samet. The quadtree and related hierarchical data structures. *Adv. Comp. Math.* , 16(2):187–260, 1984.

[36] Hanan Samet. Neighbor finding in images represented by octrees. *Comput. Vision, Graphics, Image Proc.* , 46(3):367–386, 1989.

[37] Hanan Samet. *Foundations of Multidimensional and Metric Data Structures*. Morgan Kaufmann, 2006.

[38] Bernhard Schölkopf and Alexander J. Smola. *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond*. MIT Press, 2002.

[39] Thomas W. Sederberg, Jianmin Zheng, Almaz Bakenov, and Ahmad Nasri. T-splines and T-NURCCs. *ACM Transactions on Graphics (TOG)*, 22(3):477–484, 2003.

[40] A.-V. Vuong, Carlotta Giannelli, Bert Jüttler, and Bernd Simeon. A hierarchical approach to adaptive local refinement in isogeometric analysis. *Comput. Methods Appl. Mech. Engrg.*, 200(49-52):3554–3567, 2011.